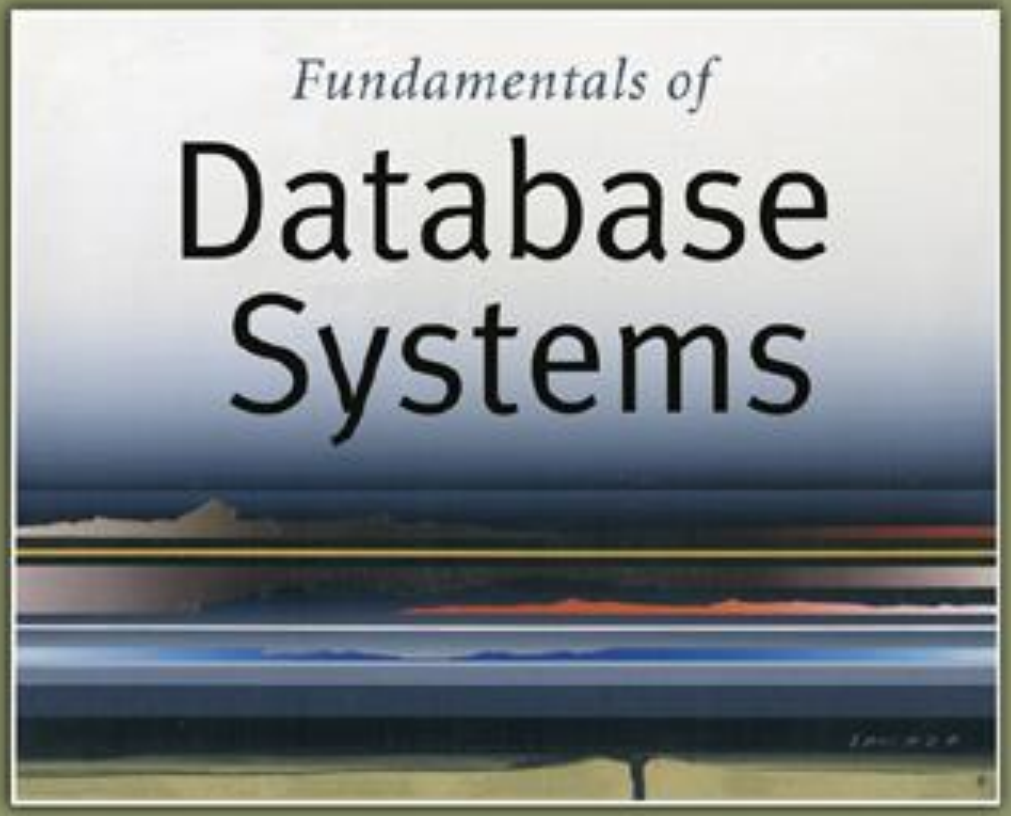


5th Edition

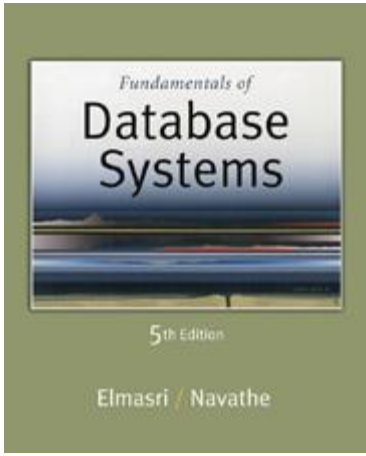
Elmasri / Navathe

Chapter 3

Data Modeling Using the Entity-Relationship (ER) Model

+

SQL Basics



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Chapter Outline

- Overview of Database Design Process
- Example Database Application (COMPANY)
- ER Model Concepts
  - Entities and Attributes
  - Entity Types, Value Sets, and Key Attributes
  - Relationships and Relationship Types
  - Weak Entity Types
  - Roles and Attributes in Relationship Types
- ER Diagrams - Notation
- ER Diagram for COMPANY Schema
- Alternative Notations – UML class diagrams, others

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Overview of Database Design Process

- Two main activities:
  - Database design
  - Applications design
- Focus in this chapter on database design
  - To design the conceptual schema for a database application
- Applications design focuses on the programs and interfaces that access the database
  - Generally considered part of software engineering

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Overview of Database Design Process

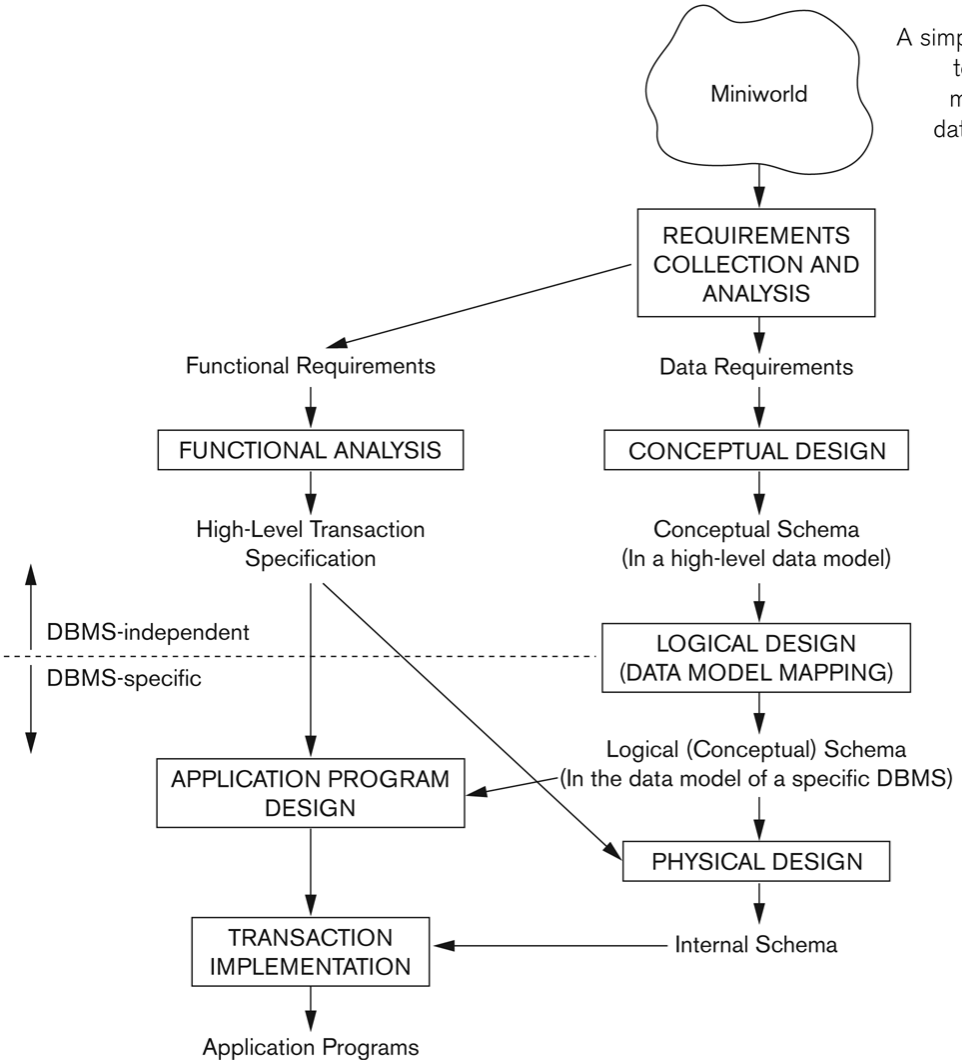


Figure 3.1
A simplified diagram to illustrate the main phases of database design.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Writing Exercise

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example COMPANY Database

- We need to create a database schema design based on the following (simplified) **requirements** of the COMPANY Database:
  - The company is organized into DEPARTMENTs. Each department has a name, number and an employee who *manages* the department. We keep track of the start date of the department manager. A department may have several locations.
  - Each department *controls* a number of PROJECTs. Each project has a unique name, unique number and is located at a single location.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example COMPANY Database (Contd.)

- We store each EMPLOYEE's social security number, address, salary, gender, and birthdate.
  - Each employee *works for* one department but may *work on* several projects.
  - We keep track of the number of hours per week that an employee currently works on each project.
  - We also keep track of the *direct supervisor* of each employee.
- Each employee may *have* a number of DEPENDENTs.
  - For each dependent, we keep track of their name, gender, birthdate, and relationship to the employee.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

ER Model Concepts

- Entities and Attributes
  - Entities are specific objects or things in the mini-world that are represented in the database.
    - For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT
  - Attributes are properties used to describe an entity.
    - For example an EMPLOYEE entity may have the attributes Name, SSN, Address, Gender, BirthDate
  - A specific entity will have a value for each of its attributes.
    - For example a specific employee entity may have Name='John Smith', SSN='123456789', Address ='731, Fondren, Houston, TX', Gender='M', BirthDate='09-JAN-55'
  - Each attribute has a *value set* (or data type) associated with it – e.g. integer, string, subrange, enumerated type, …

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Types of Attributes (1)

- Simple
  - Each entity has a single atomic value for the attribute. For example, SSN or Gender.
- Composite
  - The attribute may be composed of several components. For example:
    - Address(Apt#, House#, Street, City, State, ZipCode, Country), or
    - Name(FirstName, MiddleName, LastName).
    - Composition may form a hierarchy where some components are themselves composite.
- Multi-valued
  - An entity may have multiple values for that attribute. For example, Color of a CAR or PreviousDegrees of a STUDENT.
    - Denoted as {Color} or {PreviousDegrees}.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Types of Attributes (2)

- In general, composite and multi-valued attributes may be nested arbitrarily to any number of levels, although this is rare.
  - For example, PreviousDegrees of a STUDENT is a composite multi-valued attribute denoted by {PreviousDegrees (College, Year, Degree, Field)}
  - Multiple PreviousDegrees values can exist
  - Each has four subcomponent attributes:
    - College, Year, Degree, Field

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example of a composite attribute

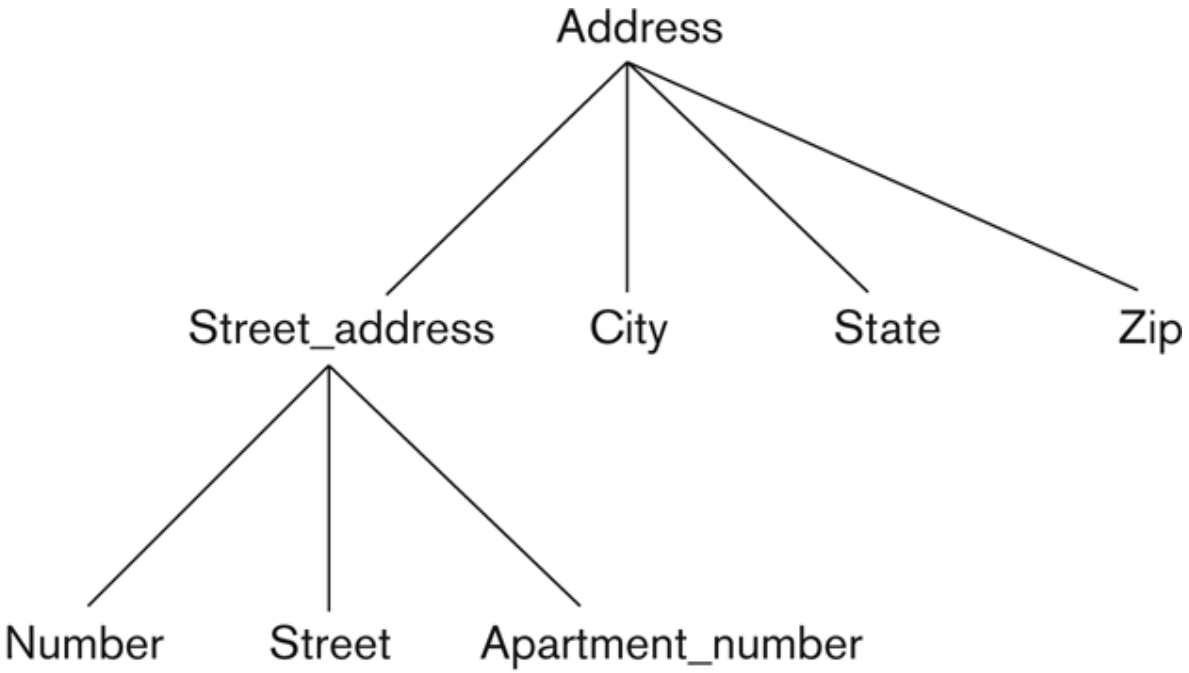


Figure 3.4
A hierarchy of composite attributes.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Entity Types and Key Attributes (1)

- Entities with the same basic attributes are grouped or typed into an entity type.
  - For example, the entity type EMPLOYEE and PROJECT.
- An attribute of an entity type for which each entity must have a unique value is called a key attribute of the entity type.
  - For example, SSN of EMPLOYEE.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Entity Types and Key Attributes (2)

- A key attribute may be composite.
  - VehicleTagNumber is a key of the CAR entity type with components (Number, State).
- An entity type may have more than one key.
  - The CAR entity type may have two keys:
    - VehicleIdentificationNumber (popularly called VIN)
    - VehicleTagNumber (Number, State), aka license plate number.
- Each key is underlined

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Displaying an Entity type

- In ER diagrams, an entity type is displayed in a rectangular box
- Attributes are displayed in ovals
  - Each attribute is connected to its entity type
  - Components of a composite attribute are connected to the oval representing the composite attribute
  - Each key attribute is underlined
  - Multivalued attributes displayed in double ovals
- See CAR example on next slide

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Entity Type CAR with two keys and a corresponding Entity Set

(a)

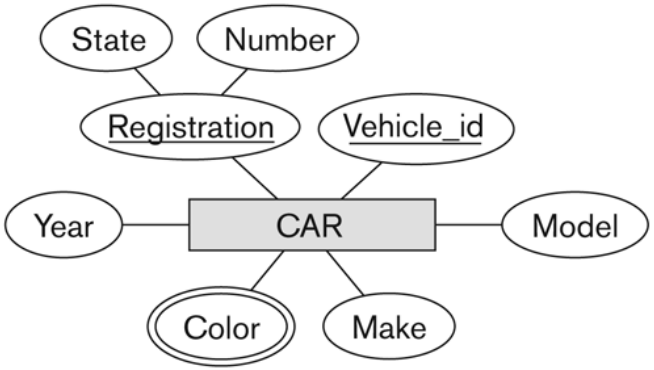


Figure 3.7
The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

(b)

CAR
Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR_1
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR_2
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR_3
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Lets do Some Hands-on

- Practice on your Systems Now along with me



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Basic SQL

- SQL language
  - Considered one of the major reasons for the commercial success of relational databases
- **SQL**
  - **Structured Query Language**
  - Statements for data definitions, queries, and updates (both DDL and DML)
  - **Core specification**
  - Plus specialized **extensions**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

SQL Data Definition and Data Types

- Terminology:
  - **Table**, **row**, and **column** used for relational model terms relation, tuple, and attribute
- `CREATE` statement
  - Main SQL command for data definition

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Attribute Data Types and Domains in SQL

- Basic **data types**
  - **Numeric** data types
    - Integer numbers: `INTEGER`, `INT`, and `SMALLINT`
    - Floating-point (real) numbers: `FLOAT` or `REAL`, and `DOUBLE PRECISION`
  - **Character-string** data types
    - Fixed length: `CHAR(`*n*`)`, `CHARACTER(`*n*`)`
    - Varying length: `VARCHAR(`*n*`)`, `CHAR VARYING(`*n*`)`, `CHARACTER VARYING(`*n*`)`

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Attribute Data Types and Domains in SQL (cont'd.)

- **Bit-string** data types
  - Fixed length: `BIT(`*n*`)`
  - Varying length: `BIT VARYING(`*n*`)`
- **Boolean** data type
  - Values of `TRUE` or `FALSE` or `NULL`
- **DATE** data type
  - Ten positions
  - Components are `YEAR`, `MONTH`, and `DAY` in the form YYYY-MM-DD

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Attribute Data Types and Domains in SQL (cont'd.)

- Additional data types
  - **Timestamp** data type (`TIMESTAMP`)
    - Includes the `DATE` and `TIME` fields
    - Plus a minimum of six positions for decimal fractions of seconds
    - Optional `WITH TIME ZONE` qualifier
  - **`INTERVAL`** data type
    - Specifies a relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Data Definition, Constraints, and Schema Changes

- Used to CREATE, DROP, and ALTER the descriptions of the tables (relations) of a database

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

CREATE TABLE

- Specifies a new base relation by giving it a name, and specifying each of its attributes and their data types (INTEGER, FLOAT, DECIMAL(i,j), CHAR(n), VARCHAR(n))
- A constraint NOT NULL may be specified on an attribute

```
CREATE TABLE DEPARTMENT (
    DNAME           VARCHAR(10)     NOT NULL,
    DNUMBER         INTEGER         NOT NULL,
    MGRSSN          CHAR(9),
    MGRSTARTDATE    CHAR(9)   );
```

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

CREATE TABLE

- In SQL2, can use the CREATE TABLE command for specifying the primary key attributes, secondary keys, and referential integrity constraints (foreign keys).
- Key attributes can be specified via the PRIMARY KEY and UNIQUE phrases

```
CREATE TABLE DEPT (
  DNAME            VARCHAR(10)    NOT NULL,
  DNUMBER          INTEGER        NOT NULL,
  MGRSSN           CHAR(9),
  MGRSTARTDATE     CHAR(9),
  PRIMARY KEY (DNUMBER),
  UNIQUE (DNAME),
  FOREIGN KEY (MGRSSN) REFERENCES EMP  );
```

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

DROP TABLE

- Used to remove a relation (base table) and its definition
- The relation can no longer be used in queries, updates, or any other commands since its description no longer exists
- Example:

```
DROP TABLE  DEPENDENT;
```

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

ALTER TABLE

- Used to add an attribute to one of the base relations
  - The new attribute will have NULLs in all the tuples of the relation right after the command is executed; hence, the NOT NULL constraint is not allowed for such an attribute
- Example:

```
ALTER TABLE EMPLOYEE ADD JOB
VARCHAR(12);
```

- The database users must still enter a value for the new attribute JOB for each EMPLOYEE tuple.
  - This can be done using the UPDATE command.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Lets Get Back to Conceptual Discussion



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

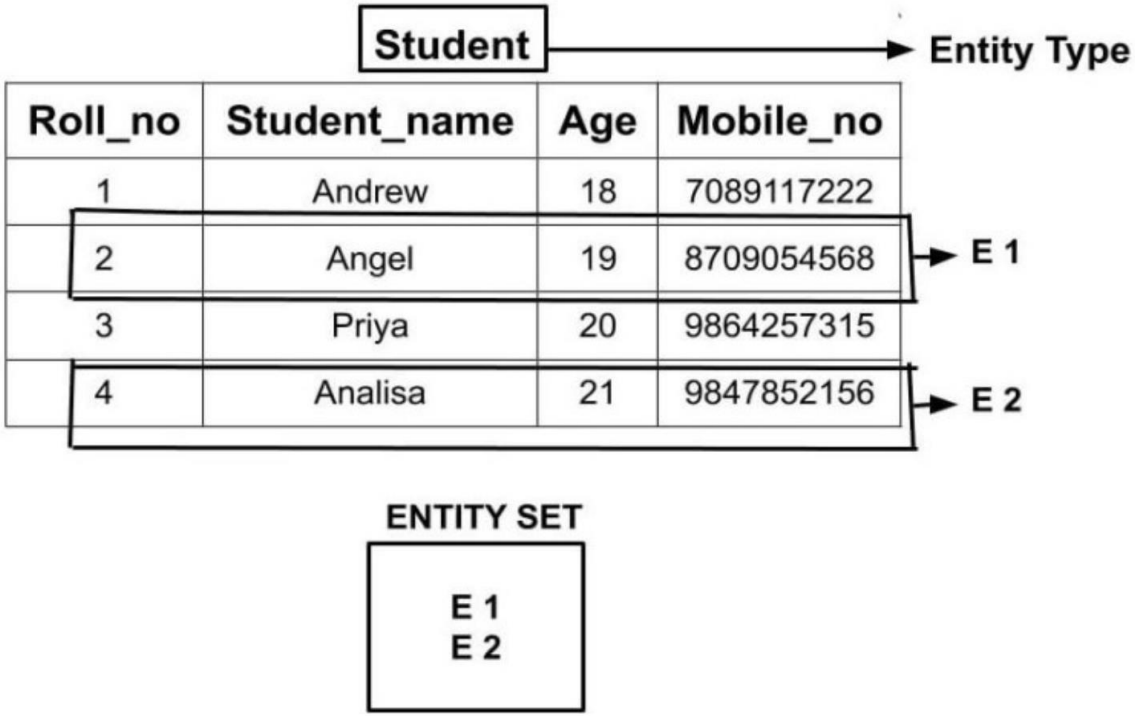
Entity Set

- Each entity type will have a collection of entities stored in the database
  - Called the **entity set**
- Previous slide shows three CAR entity instances in the entity set for CAR
- Same name (CAR) used to refer to both the entity type and the entity set
- Entity set is the current *state* of the entities of that type that are stored in the database

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Entity Set

- Entity Set is a collection of entities of the same entity type



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Initial Design of Entity Types for the COMPANY Database Schema

- Based on the requirements, we can identify four initial entity types in the COMPANY database:
  - DEPARTMENT
  - PROJECT
  - EMPLOYEE
  - DEPENDENT
- Their initial design is shown on the following slide
- The initial attributes shown are derived from the requirements description

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Initial Design of Entity Types:
EMPLOYEE, DEPARTMENT, PROJECT, DEPENDENT

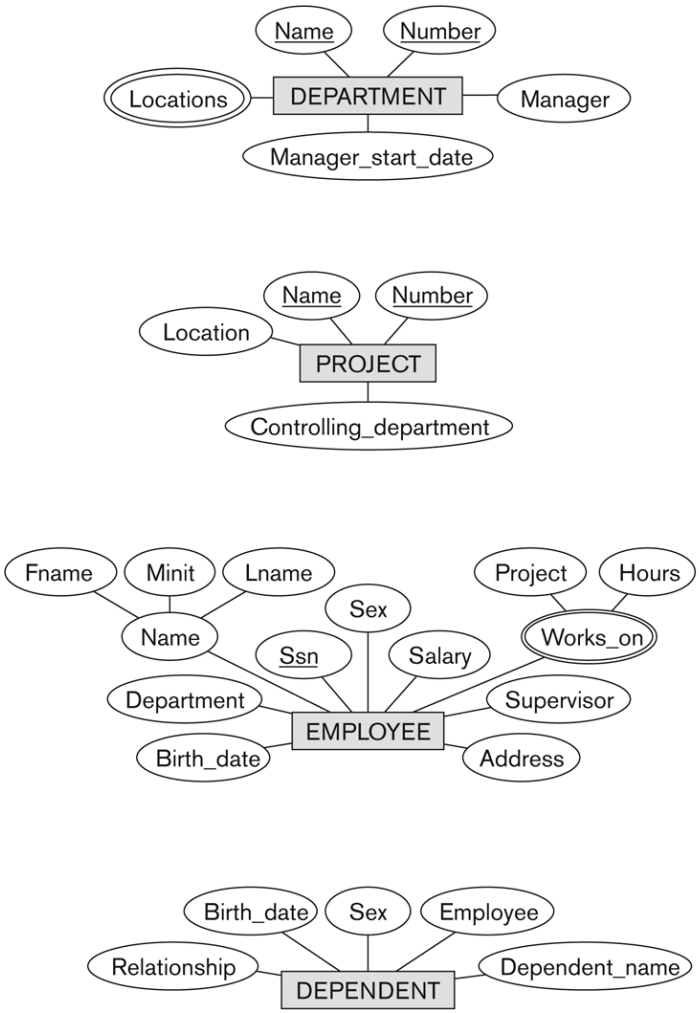


Figure 3.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Refining the initial design by introducing **relationships**

- The initial design is typically not complete
- Some aspects in the requirements will be represented as **relationships**
- ER model has three main concepts:
  - Entities (and their entity types and entity sets)
  - Attributes (simple, composite, multivalued)
  - Relationships (and their relationship types and relationship sets)
- We introduce **relationship** concepts next

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationships

- Introduction to Relationship Types +
- Relationship types and Instances +
- Role names and Relation types +
- Role names and Recursive Relationship types +
- Constraints on Relationship Types +
- Attributes of Relationship Types +
- Relationship Representation in ER Diagram +
- Higher Degree Relationships +
- Examples +

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Introduction to Relationship Type

- By reviewing the initial design of the COMPANY DB example we can see the following:
  - Some attributes of an entity reference other attributes in the same or other entities. For example, The attribute Department of the EMPLOYEE entity type refers to the DEPARTMENT entity that the employee works for.
  - These are implicit relationships that exist between entity types.
  - A better representation will be obtained if these are represented as relationships between the entity types rather than linking attributes.
- Here we will describe another major component of ER model; namely relationships.
- We will discuss relationship types, instances, degrees, and constraints on relationships.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationship Types and Instances…

- Informally, a relationship type is a meaningful association among entity types.
- A relationship (instance) is an association of entities where the association includes one entity from each participating entity type.
- Mathematically, a relationship type (set) R between entity types E_1, E_2, …, En is a set of relationship instance r_i, where each r_i associates n individual entities (e_1, e_2, …, e_n); and each entity e_i, in r_i, is a member of the entity type E_j, $1 <= j <= n$.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example

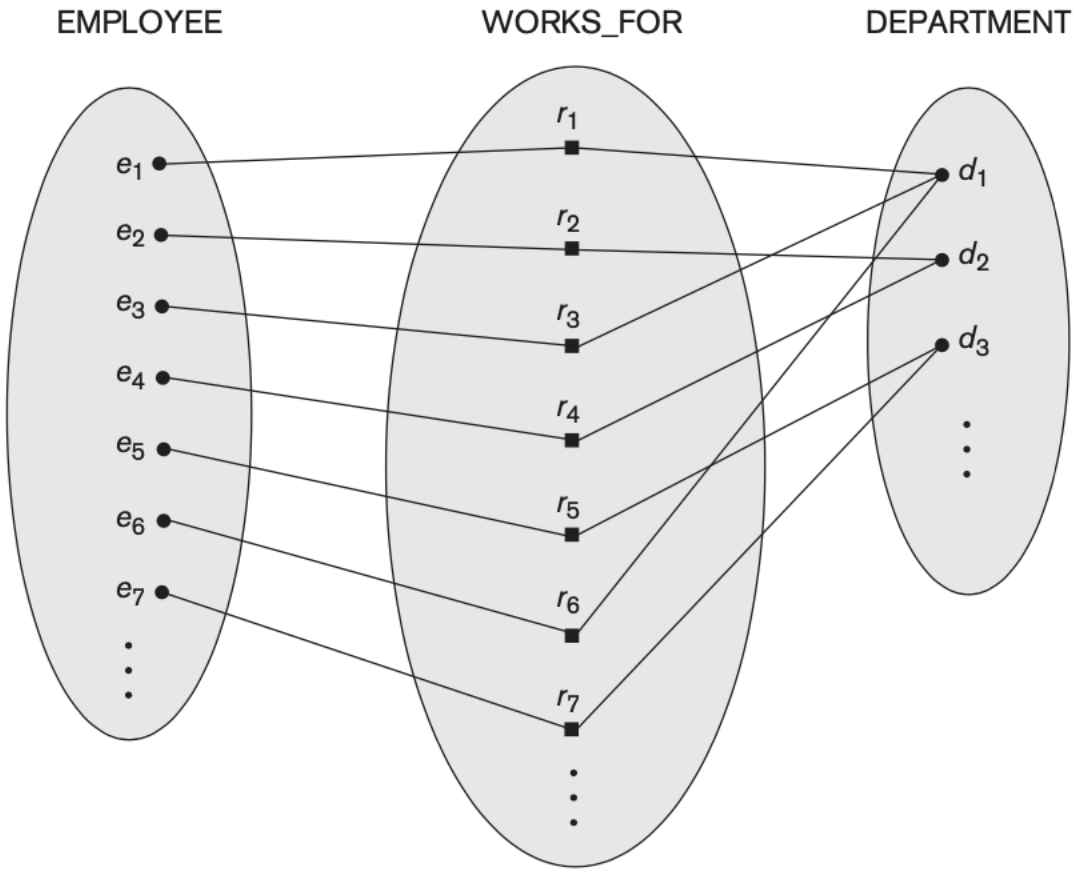


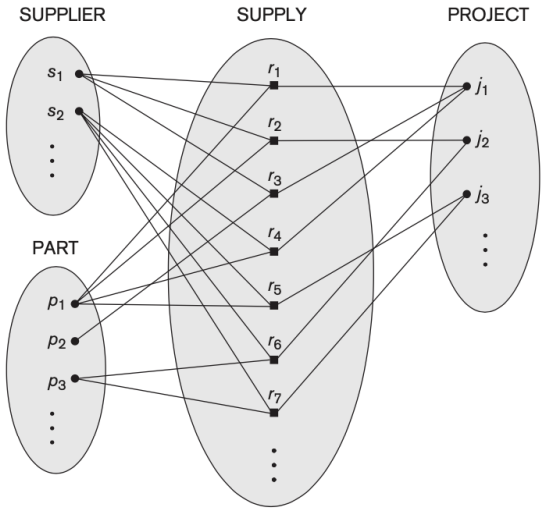
Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Degree of a Relationship Type

- The **degree** of a relationship type is the number of participating entity types
  - A relationship type of degree one is called **unary** (to be discussed more)
  - A relationship type of degree two is called **binary**
  - A relationship type of degree two is called **ternary**

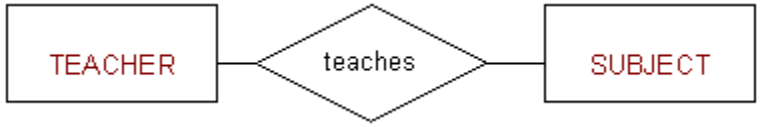
- Example of Degree Binary relationship: WORKS_FOR (previous slide)
- Example of Degree Ternary relationship: SUPPLY (below image)



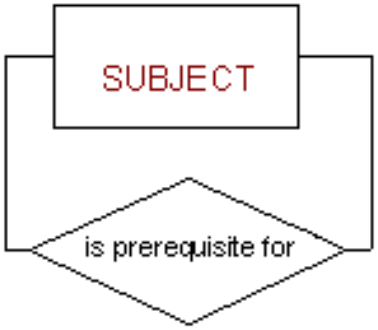
Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Degree of Relationship

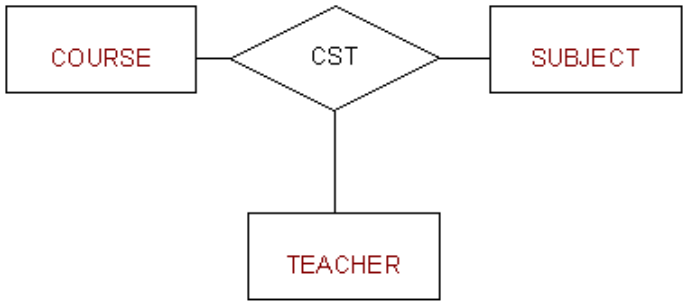
- Binary
  - Teacher teaches a subject



- Unary (to be discussed more)
  - Subjects may be prerequisites for other subjects



- Ternary
  - which teachers taught which subjects in which courses

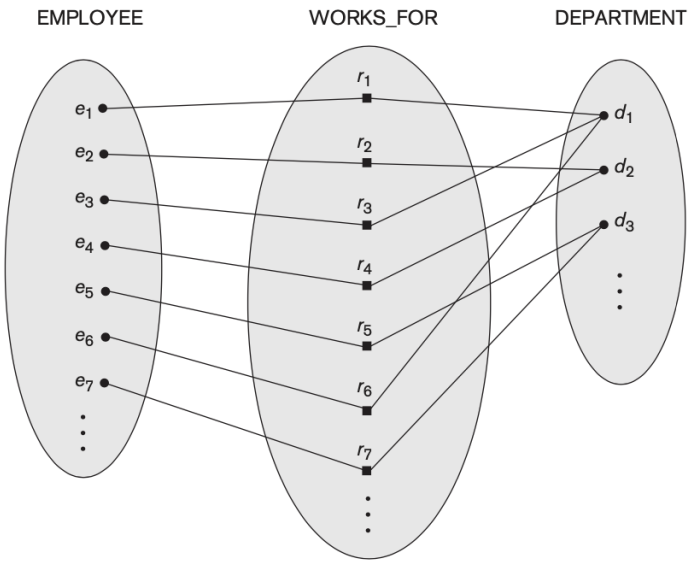


Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationships as Attributes

- Consider the WORKS_FOR relationship type
- Two Alternatives to store relationship as attribute

1. think of an attribute called Department of the EMPLOYEE entity type, where the value of Department for each EMPLOYEE entity is (a reference to) the DEPARTMENT entity for which that employee works.
    1. value set for this Department attribute is the set of *all* DEPARTMENT entities

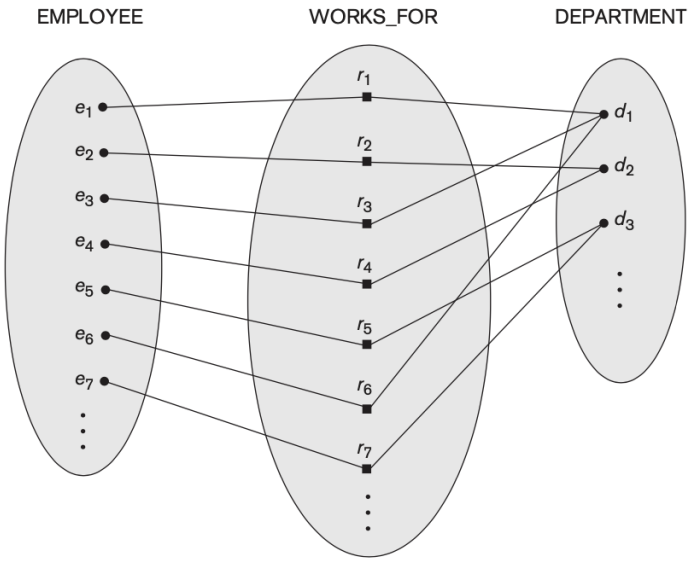


Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationships as Attributes

- Consider the WORKS_FOR relationship type
- Two Alternatives to store relationship as attribute

2. think of a multivalued attribute Employees of the entity type DEPARTMENT whose value for each DEPARTMENT entity is the set of EMPLOYEE entities who work for that department.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Lets Do Some Hands-On

- Do the Practice right now along with me



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Specifying Updates in SQL

- There are three SQL commands to modify the database: **INSERT**, **DELETE**, and **UPDATE**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

INSERT

- In its simplest form, it is used to add one or more tuples to a relation
- Attribute values should be listed in the same order as the attributes were specified in the **CREATE TABLE** command

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

INSERT (contd.)

- Example:

```
U1: INSERT INTO          EMPLOYEE
    VALUES ('Richard','K','Marini', '653298653', '30-DEC-52',
    '98 Oak Forest,Katy,TX', 'M', 37000,'987654321', 4 )
```

- An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple
  - Attributes with NULL values can be left out
- Example: Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.

```
U1A:  INSERT INTO      EMPLOYEE (FNAME, LNAME,
                                 SSN)
      VALUES ('Richard', 'Marini', '653298653')
```

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

INSERT (contd.)

- Important Note: Only the constraints specified in the DDL commands are automatically enforced by the DBMS when updates are applied to the database
  - Another variation of INSERT allows insertion of *multiple tuples* resulting from a query into a relation

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

INSERT (contd.)

- Example: Suppose we want to create a temporary table that has the name, number of employees, and total salaries for each department.
  - A table DEPTS_INFO is created by U3A, and is loaded with the summary information retrieved from the database by the query in U3B.

```
U3A:    CREATE TABLE  DEPTS_INFO
                (DEPT_NAME          VARCHAR(10),
                 NO_OF_EMPS         INTEGER,
                 TOTAL_SAL          INTEGER);

U3B:    INSERT INTO   DEPTS_INFO (DEPT_NAME,
                      NO_OF_EMPS, TOTAL_SAL)
                SELECT      DNAME, COUNT (*), SUM (SALARY)
                FROM        DEPARTMENT, EMPLOYEE
                WHERE       DNUMBER=DNO
                GROUP BY    DNAME ;
```

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

INSERT (contd.)

- Note: The DEPTS_INFO table may not be up-to-date if we change the tuples in either the DEPARTMENT or the EMPLOYEE relations *after* issuing U3B. We have to create a view (see later) to keep such a table up to date.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

DELETE

- Removes tuples from a relation
  - Includes a WHERE-clause to select the tuples to be deleted
  - Referential integrity should be enforced
  - Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)
  - A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table
  - The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

DELETE (contd.)

- Examples:

U4A:
```
DELETE FROM EMPLOYEE
WHERE LNAME='Brown’
```

U4B:
```
DELETE FROM EMPLOYEE
WHERE SSN='123456789’
```

U4C:
```
DELETE FROM EMPLOYEE
WHERE DNO IN
(SELECT DNUMBER
FROM DEPARTMENT
WHERE
DNAME='Research')
```

U4D:
```
DELETE FROM EMPLOYEE
```

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

UPDATE

- Used to modify attribute values of one or more selected tuples
- A WHERE-clause selects the tuples to be modified
- An additional SET-clause specifies the attributes to be modified and their new values
- Each command modifies tuples *in the same relation*
- Referential integrity should be enforced

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

UPDATE (contd.)

- Example: Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively.

```
U5:     UPDATE    PROJECT
        SET       PLOCATION = 'Bellaire',
                            DNUM = 5
        WHERE     PNUMBER=10
```

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

UPDATE (contd.)

- Example: Give all employees in the 'Research' department a 10% raise in salary.

```
U6: UPDATE    EMPLOYEE
    SET       SALARY = SALARY *1.1
    WHERE     DNO  IN (SELECT    DNUMBER
                  FROM       DEPARTMENT
                  WHERE      DNAME='Research')
```

- In this request, the modified SALARY value depends on the original SALARY value in each tuple
  - The reference to the SALARY attribute on the right of = refers to the old SALARY value before modification
  - The reference to the SALARY attribute on the left of = refers to the new SALARY value after modification

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Recap of SQL Queries

- A query in SQL can consist of up to six clauses, but only the first two, SELECT and FROM, are mandatory. The clauses are specified in the following order:

SELECT <attribute list>
FROM <table list>
[**WHERE** <condition>]
[**GROUP BY** <grouping attribute(s)>]
[**HAVING** <group condition>]
[**ORDER BY** <attribute list>]

- There are three SQL commands to modify the database: **INSERT**, **DELETE**, and **UPDATE**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Retrieval Queries in SQL

- SQL has one basic statement for retrieving information from a database; the **SELECT** statement
  - This is *not the same as* the SELECT operation of the relational algebra
- Important distinction between SQL and the formal relational model:
  - SQL allows a table (relation) to have two or more tuples that are identical in all their attribute values
  - Hence, an SQL relation (table) is a **multi-set** (sometimes called a **bag**) of tuples; it is *not* a set of tuples
- SQL relations can be constrained to be sets by specifying PRIMARY KEY or UNIQUE attributes, or by using the DISTINCT option in a query

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Retrieval Queries in SQL (contd.)

- A **bag** or **multi-set** is like a set, but an element may appear more than once.
  - Example: {A, B, C, A} is a bag. {A, B, C} is also a bag that also is a set.
  - Bags also resemble lists, but the order is irrelevant in a bag.
- Example:
  - {A, B, A} = {B, A, A} as bags
  - However, [A, B, A] is not equal to [B, A, A] as lists

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Retrieval Queries in SQL (contd.)

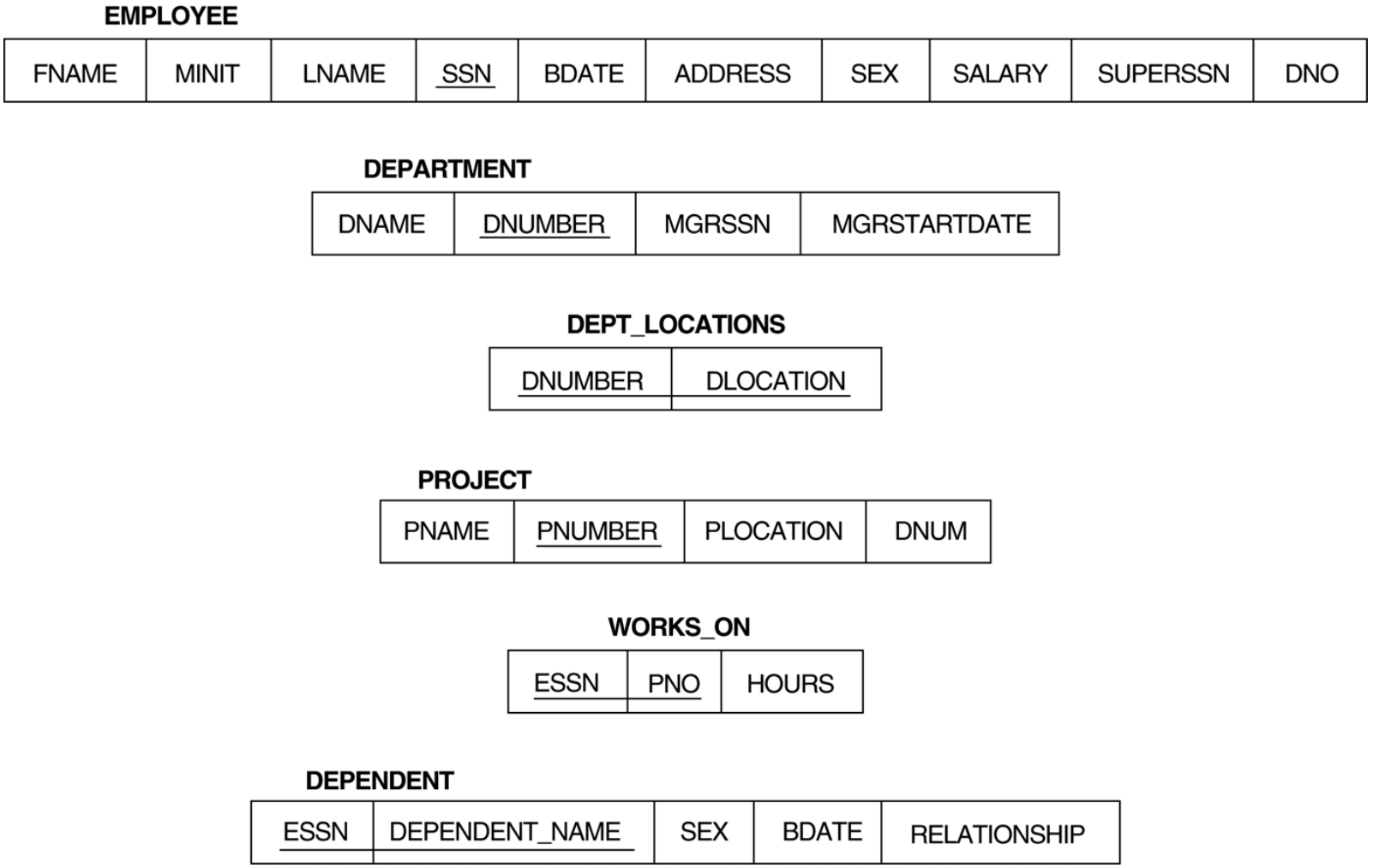
- Basic form of the SQL SELECT statement is called a *mapping* or a SELECT-FROM-WHERE *block*

SELECT <attribute list>
FROM <table list>
WHERE <condition>

  - <attribute list> is a list of attribute names whose values are to be retrieved by the query
  - <table list> is a list of the relation names required to process the query
  - <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relational Database Schema--Figure 5.5



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Populated Database--Fig.5.6

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
	5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Simple SQL Queries

- Basic SQL queries correspond to using the following operations of the relational algebra:
  - SELECT
  - PROJECT
  - JOIN
- All subsequent examples use the COMPANY database

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Simple SQL Queries (contd.)

- Example of a simple query on one relation
- Query 0: Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.

```
Q0: SELECT    BDATE, ADDRESS
    FROM      EMPLOYEE
    WHERE     FNAME='John' AND MINIT='B'
    AND       LNAME='Smith'
```

  - Similar to a SELECT-PROJECT pair of relational algebra operations:
    - The SELECT-clause specifies the projection attributes and the WHERE-clause specifies the selection condition
  - However, the result of the query may contain duplicate tuples

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Simple SQL Queries (contd.)

- Query 1: Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1:SELECT    FNAME, LNAME, ADDRESS
   FROM      EMPLOYEE, DEPARTMENT
   WHERE     DNAME='Research' AND DNUMBER=DNO
```

  - Similar to a SELECT-PROJECT-JOIN sequence of relational algebra operations
  - (DNAME='Research') is a selection condition (corresponds to a SELECT operation in relational algebra)
  - (DNUMBER=DNO) is a join condition (corresponds to a JOIN operation in relational algebra)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Simple SQL Queries (contd.)

- Query 2: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

```
Q2: SELECT    PNUMBER, DNUM, LNAME, BDATE, ADDRESS
          FROM        PROJECT, DEPARTMENT, EMPLOYEE
          WHERE       DNUM=DNUMBER AND MGRSSN=SSN
                           AND PLOCATION='Stafford'
```

  - In Q2, there are two join conditions
  - The join condition DNUM=DNUMBER relates a project to its controlling department
  - The join condition MGRSSN=SSN relates the controlling department to the employee who manages that department

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Lets Go Back to Conceptual Discussion

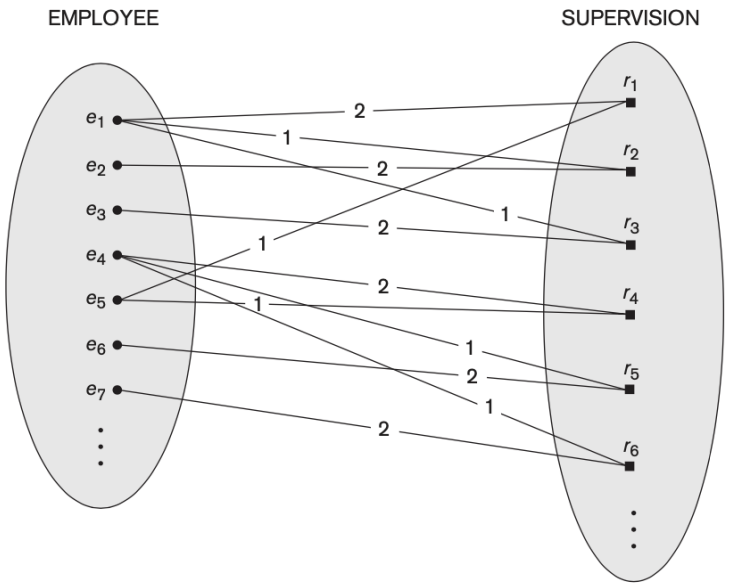
- Remaining Topics



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Recursive Relationship and Role Names

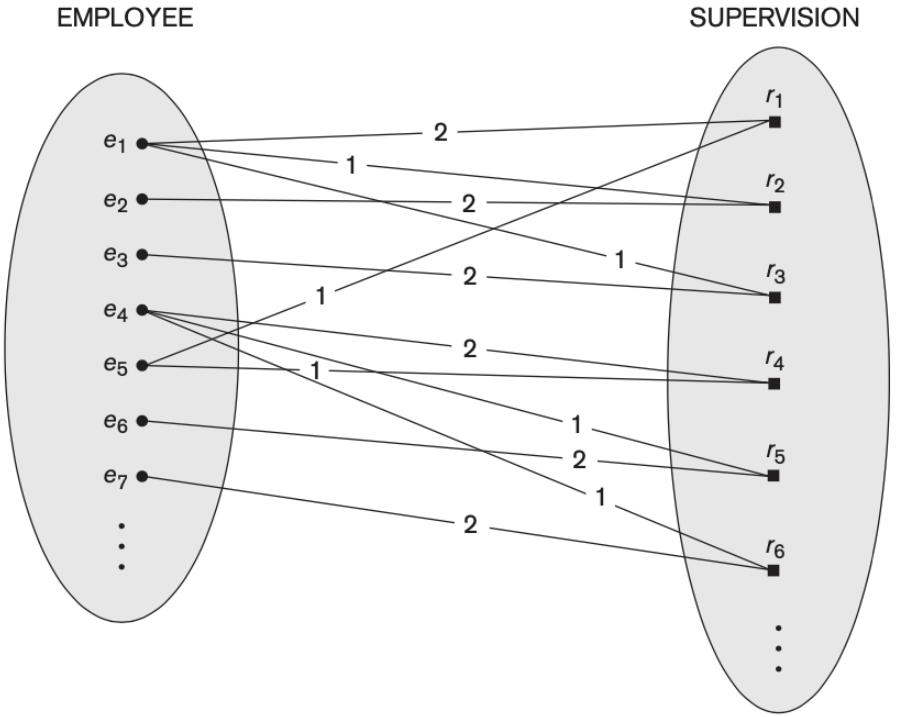
- in some cases the *same* entity type participates more than once in a relationship type in *different roles*.
- In such cases the role name becomes essential for distinguishing the meaning of the role that each participating entity plays
- The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity set
  - the EMPLOYEE entity type *participates twice* in SUPERVISION: once in the role of *supervisor* (or *boss*), and once in the role of *supervisee* (or *subordinate*)



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Recursive Relationship and Role Names

- Each relationship instance *r_i* in SUPERVISION associates two different employee entities *e_j* and *e_k*, one of which plays the role of supervisor and the other the role of supervisee.
- lines marked ‘1’ represent the supervisor role, and those marked ‘2’ represent the supervisee role
- Example
  - *e*1 supervises *e*2 and *e*3
  - *e*4 supervises *e*6 and *e*7
  - *e*5 supervises *e*1 and *e*4



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Department & Employee

- Consider Employees are working in departments:
  - Department
  - Employee
  - Manager/ Boss??

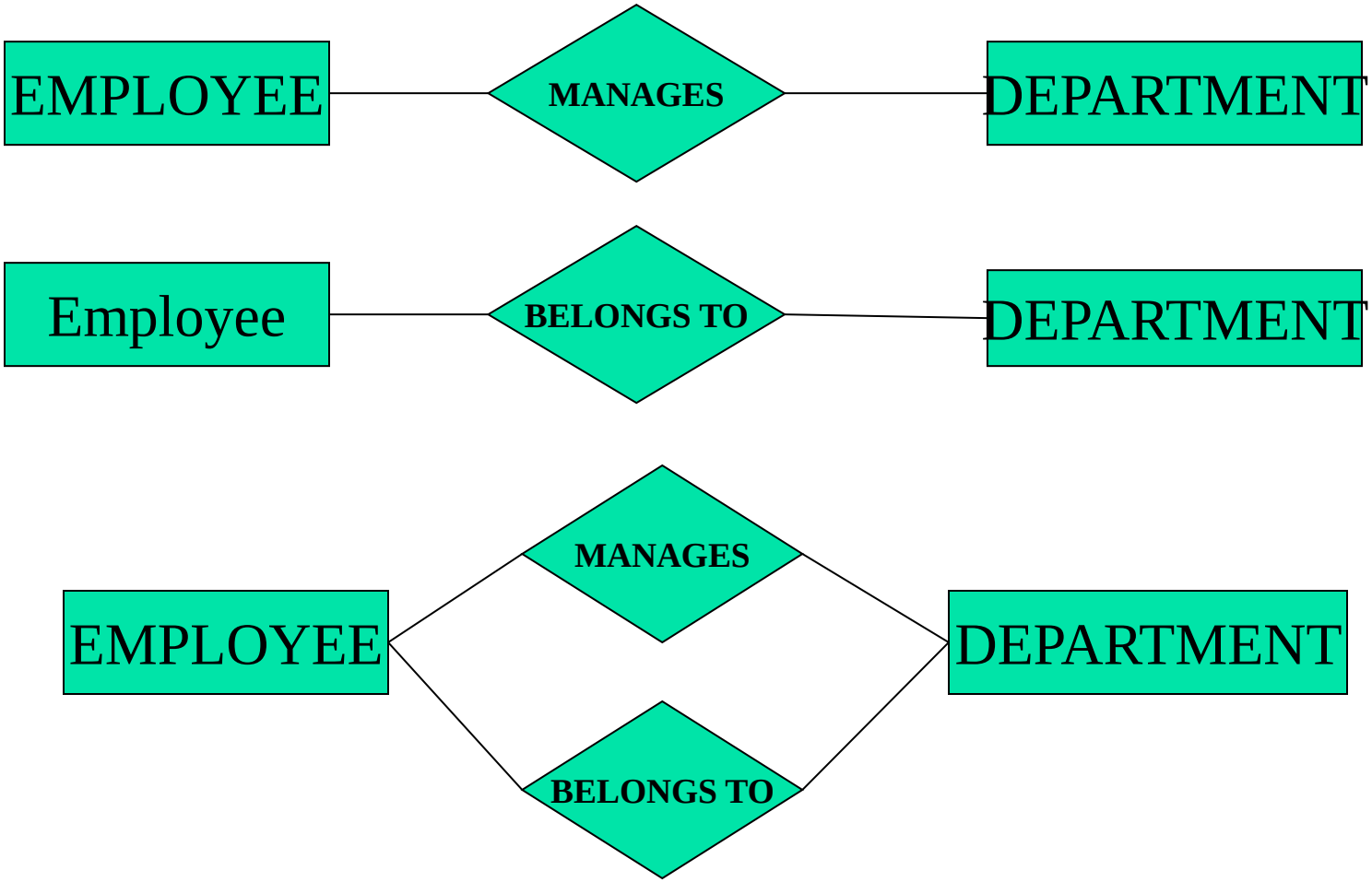
Entities

  - Employees are belong to Departments
  - Some employees are managers of department
  - Employee may work in more than one dept
  - Some are managers of other employees

Relationships

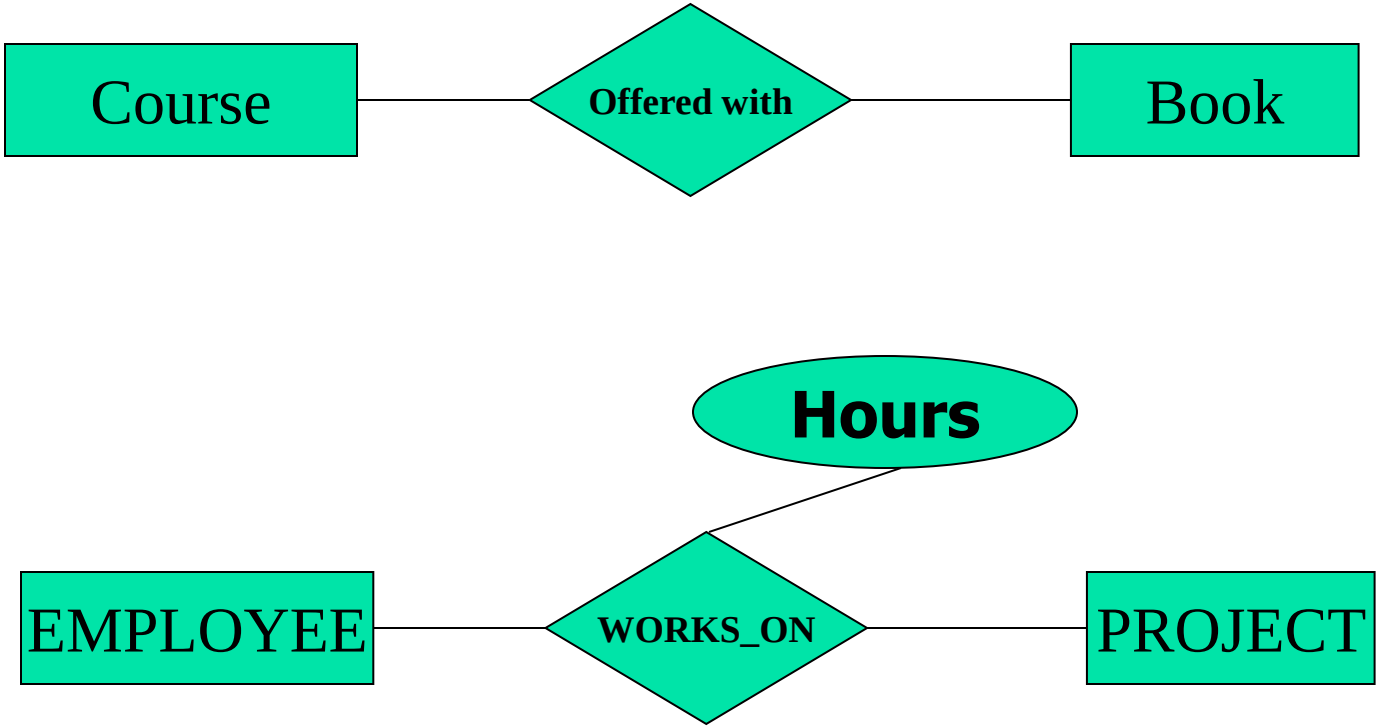
Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Basic Examples



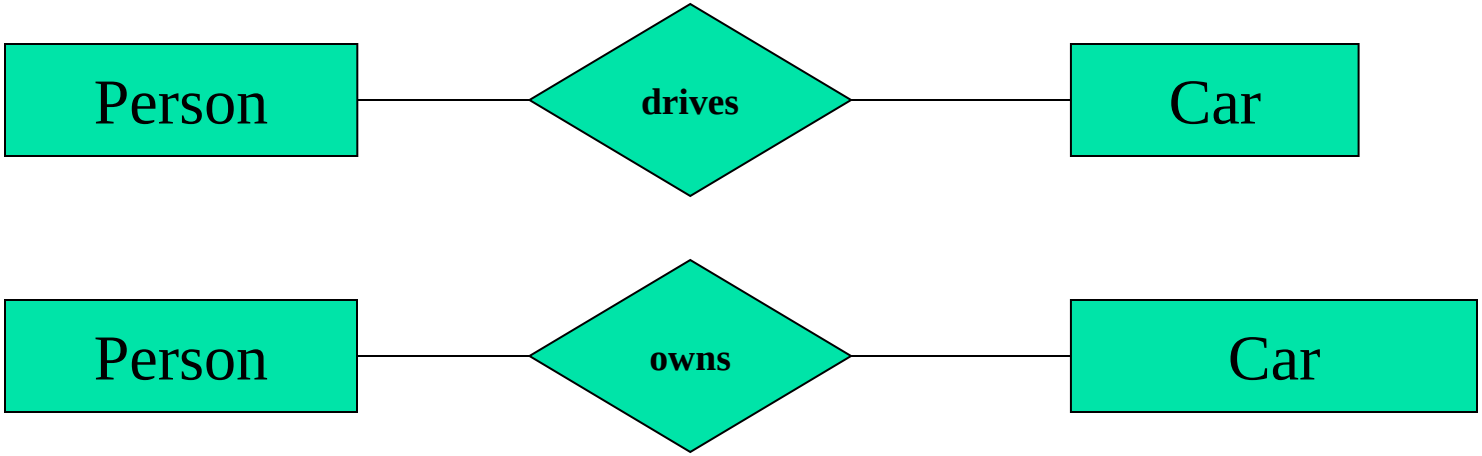
Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Basic Examples …



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

More Basic Examples



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Constraints on Binary Relationship Types

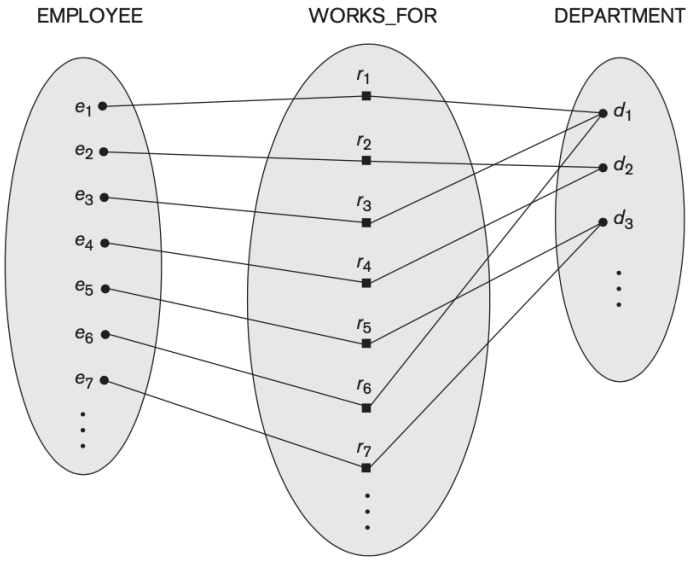
- Relationship types usually have certain constraints that limit the possible combinations of entities that may participate in the corresponding relationship set.
- Example
  - if the company has a rule that each employee must work for exactly one department, then we would like to describe this constraint in the schema
- Two main types of binary relationship constraints
  - *cardinality ratio*
  - *participation*

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Cardinality Ratios for Binary Relationships

- Specifies the *maximum* number of relationship instances that an entity can participate in
- Example
  - in the WORKS_FOR binary relationship type, DEPARTMENT:EMPLOYEE is of cardinality ratio **1:N**, meaning that each department can be related to (that is, employs) any number of employees (N), but an employee can be related to (work for) at most one department (1)

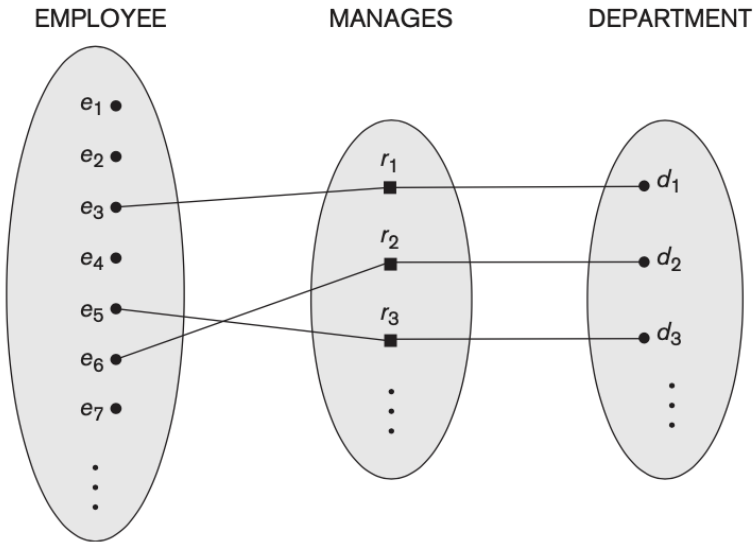
Example of 1:N relationship



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Cardinality Ratios for Binary Relationships

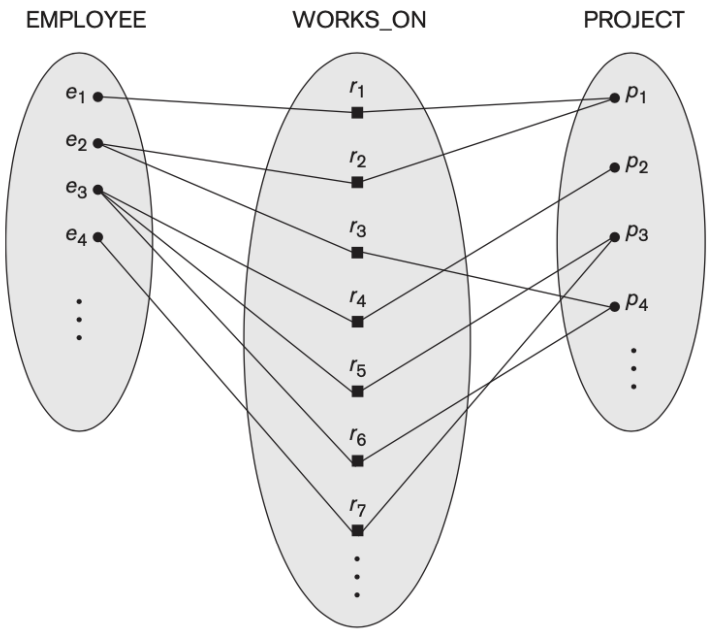
- Example of 1:1 relationship
  - relates a department entity to the employee who manages that department.
  - at any point in time—an employee can manage at most one department and a department can have at most one manager



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Cardinality Ratios for Binary Relationships

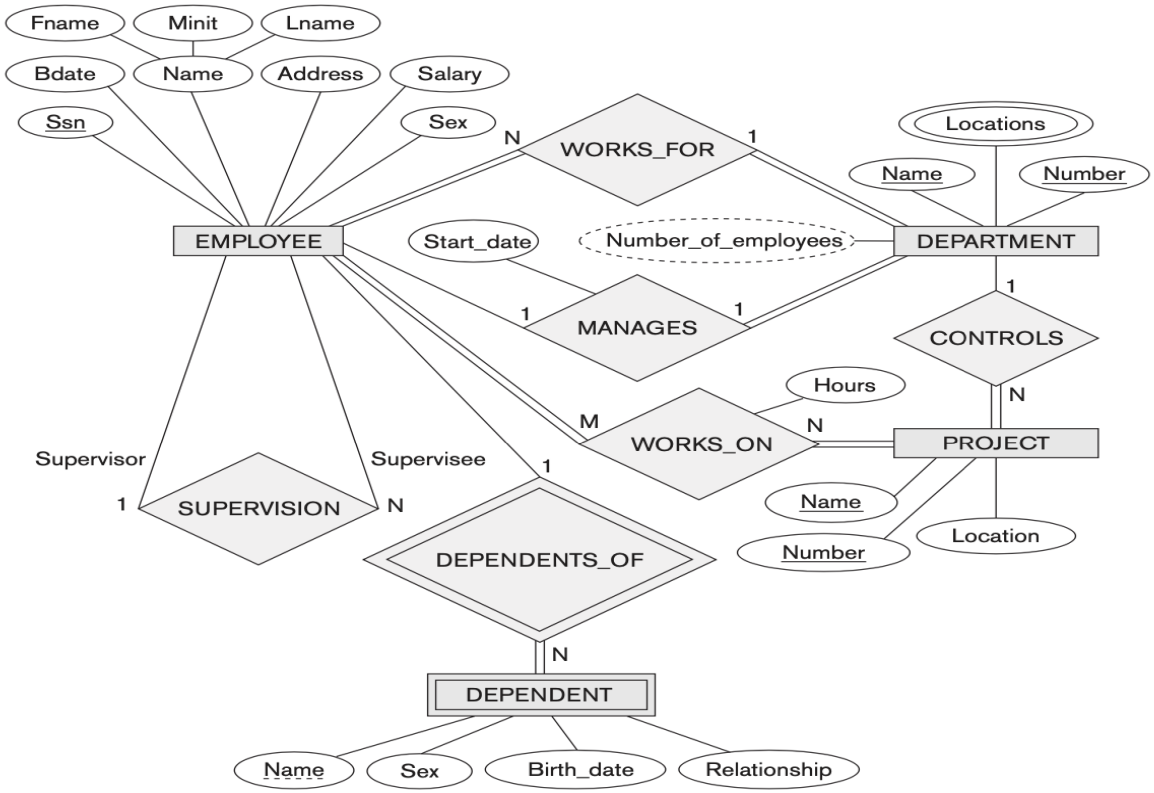
- Example of M:N relationship
  - an employee can work on several projects and a project can have several employees.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Cardinality Ratios for Binary Relationships

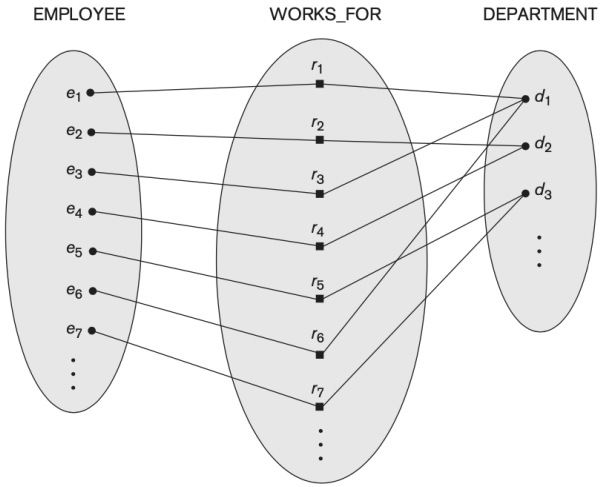
- Cardinality ratios for binary relationships are represented on ER diagrams by displaying 1, M, and N on the diamonds



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Participation Constraints and Existence Dependencies

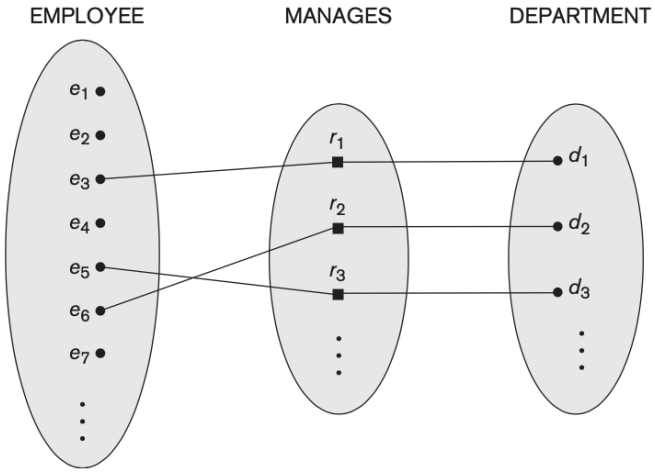
- The **participation constraint** specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
- **Two types of participation constraints: total and partial**
- **Example of Total**: If a company policy states that *every* employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance
- The participation of EMPLOYEE in WORKS_FOR is called total participation, meaning that every entity in the total set of employee entities must be related to a department entity via WORKS_FOR
- Total participation is also called **existence dependency**



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Participation Constraints and Existence Dependencies

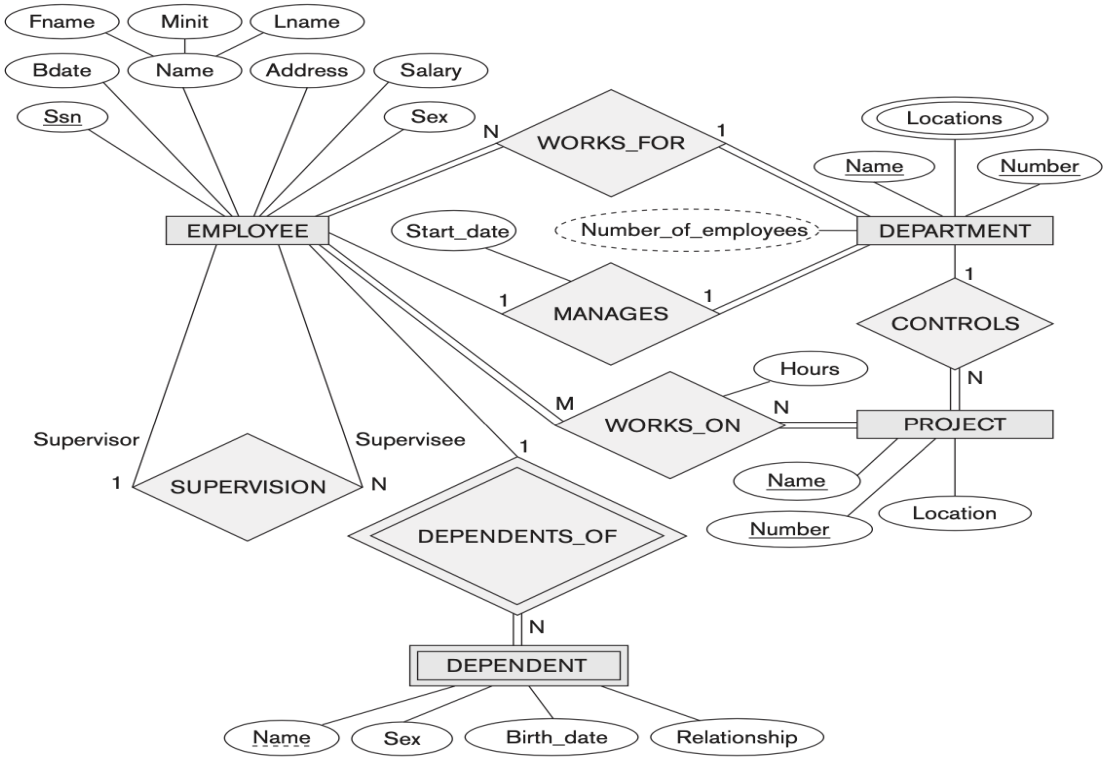
- The **participation constraint** specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
- **Two types of participation constraints: total and partial**
- **Example of Partial:** we do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is **partial**
- meaning that *some* or *part of the set of* employee entities are related to some department entity via MANAGES, but not necessarily all.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Participation Constraints and Existence Dependencies

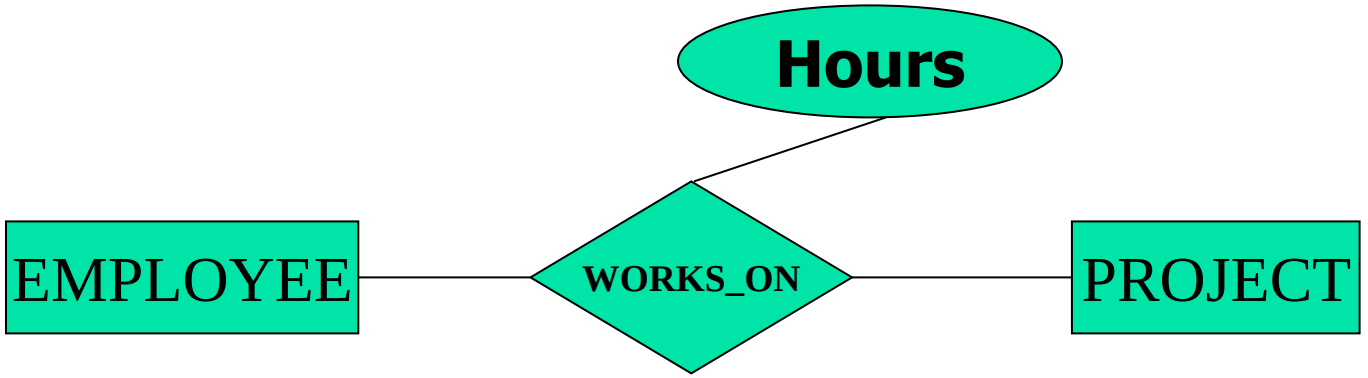
- In ER diagrams, total participation (or existence dependency) is displayed as a *double line* connecting the participating entity type to the relationship, whereas partial participation is represented by a *single line*



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Attributes of Relationship Types

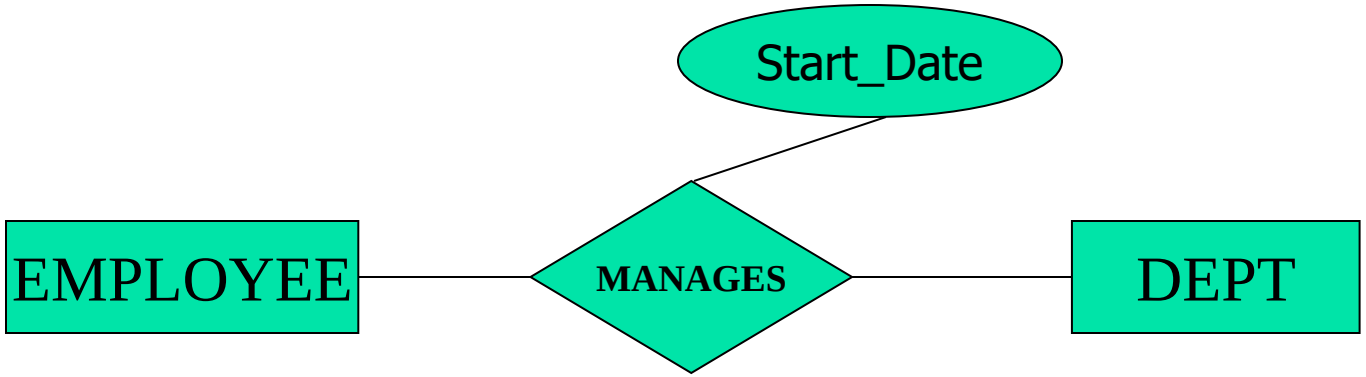
- Relationship types can also have attributes, similar to those of entity types
- **Example 1**: to record the number of hours per week that a particular employee works on a particular project



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Attributes of Relationship Types

- Relationship types can also have attributes, similar to those of entity types
- **Example 2**: date on which a manager started managing a department via an attribute Start_date for the MANAGES relationship type

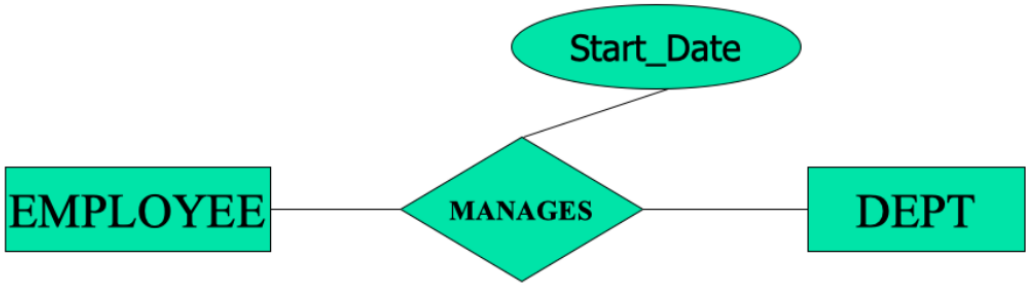


*MANAGES is a 1:1 relationship, so every department or employee entity participates in *at most one* relationship instance.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Attributes of Relationship Types - Placement

- **For 1:1**
  - Attributes can be migrated to one of the participating entity types
- **Example**
  - "Start_date" can be an attribute of either EMPLOYEE or DEPT

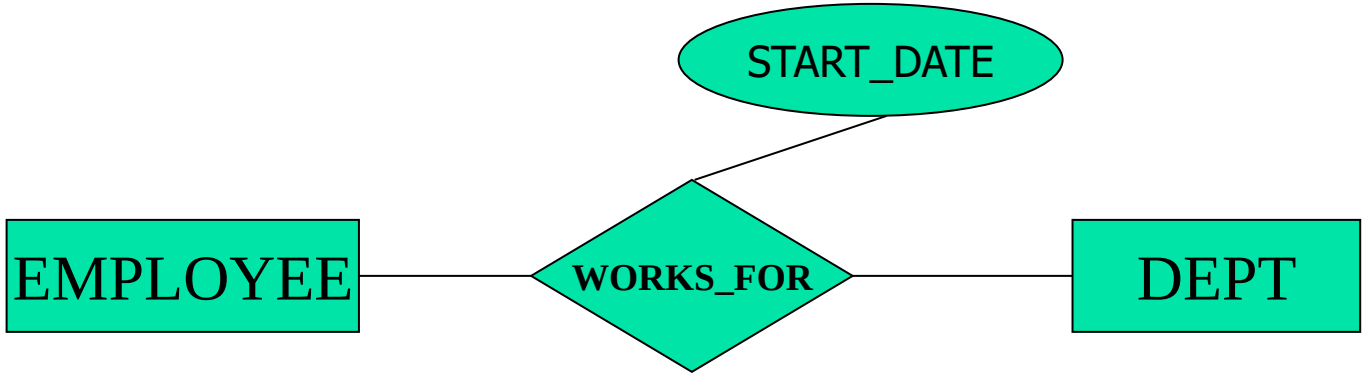


*MANAGES is a 1:1 relationship, so every department or employee entity participates in *at most one* relationship instance.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Attributes of Relationship Types - Placement

- **For 1:N**
  - a relationship attribute can be migrated only to the entity type on the N-side of the relationship
- **Example**
  - "start_date" indicates when an employee started working for a department, should be included as an attribute of EMPLOYEE

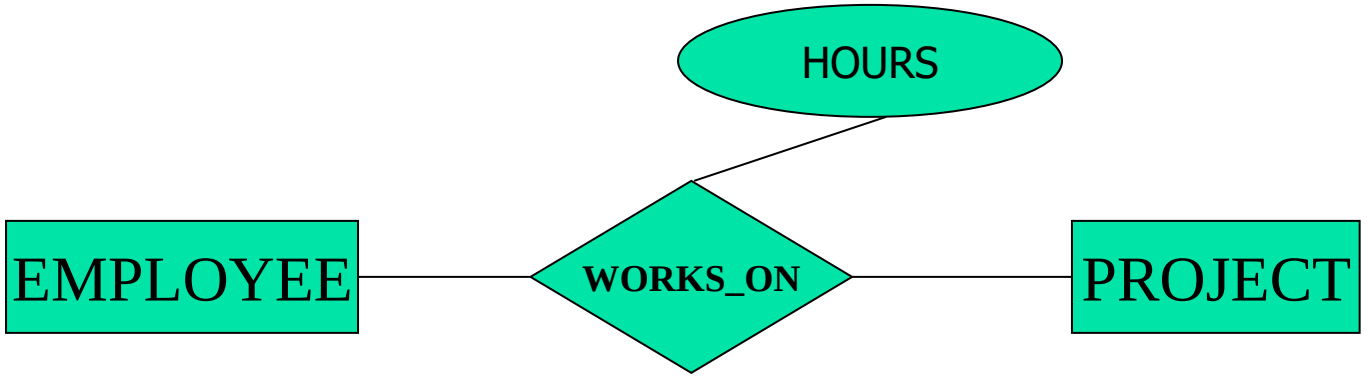


*each department can be related to any number of employees (N), but an employee can be related to at most one department (1) = N:1 relationship type

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Attributes of Relationship Types - Placement

- **For M:N**
  - a relationship attribute is determined by combination of participating entities and not by single entity
- **Example:**
  - number of hours per week an employee currently works on a project is determined by an employee-project combination and not separately by either entity.



*each employee can work on any number of projects (M), and a project can have any number of employees (N) = M:N relationship type

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Weak Entity Types

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

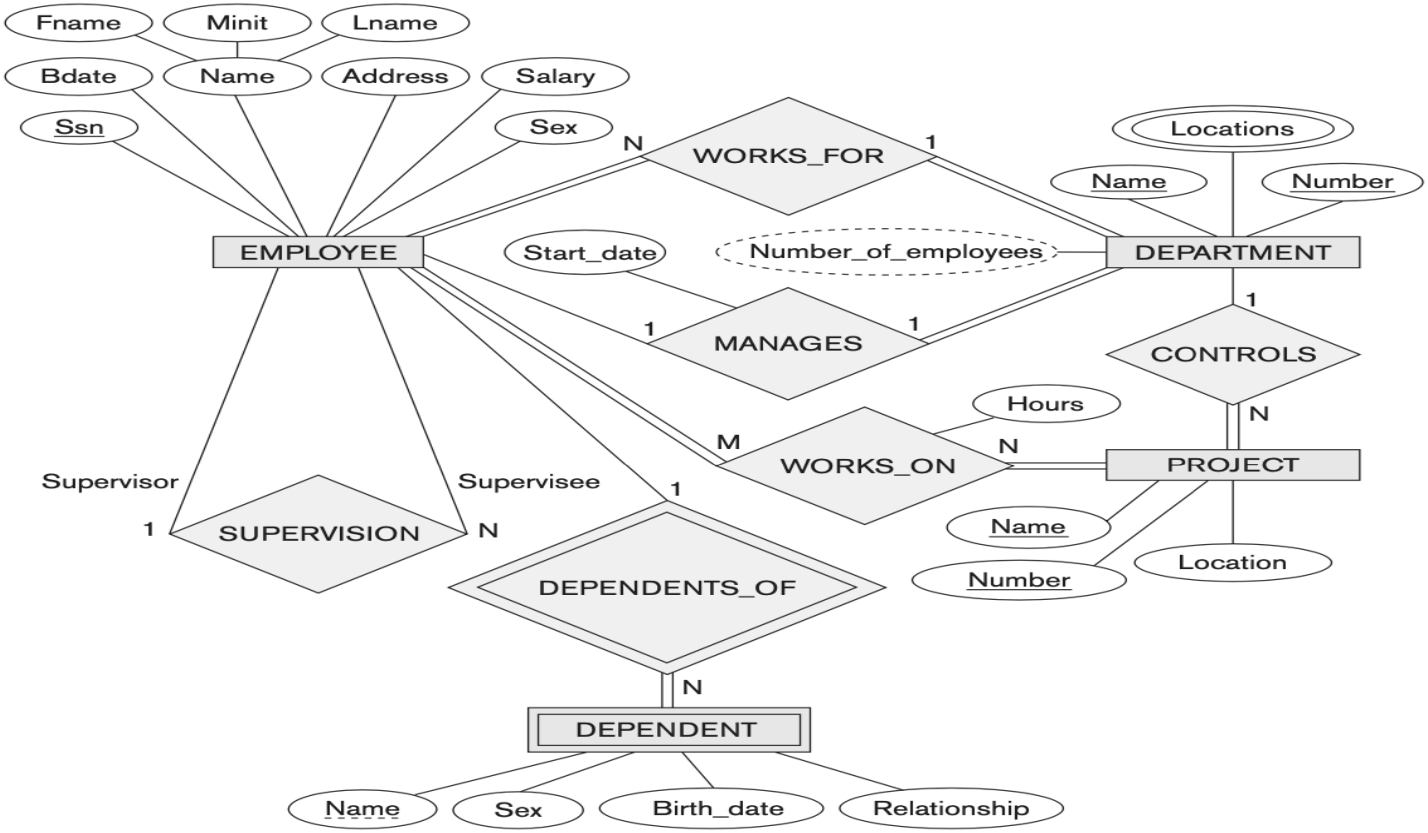
Weak Entity Types

- Definition
  - Entity types that do not have key attributes of their own
  - They are identified by being related to specific entities from another (strong/owner) entity type
- Identifying Relationship
  - A relationship type that relates a weak entity type to its owner/strong entity type
- Participation Constraint (existence dependency)
  - always has a *total participation constraint* because it cannot be identified without an owner entity.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Weak Entity Types - Representation

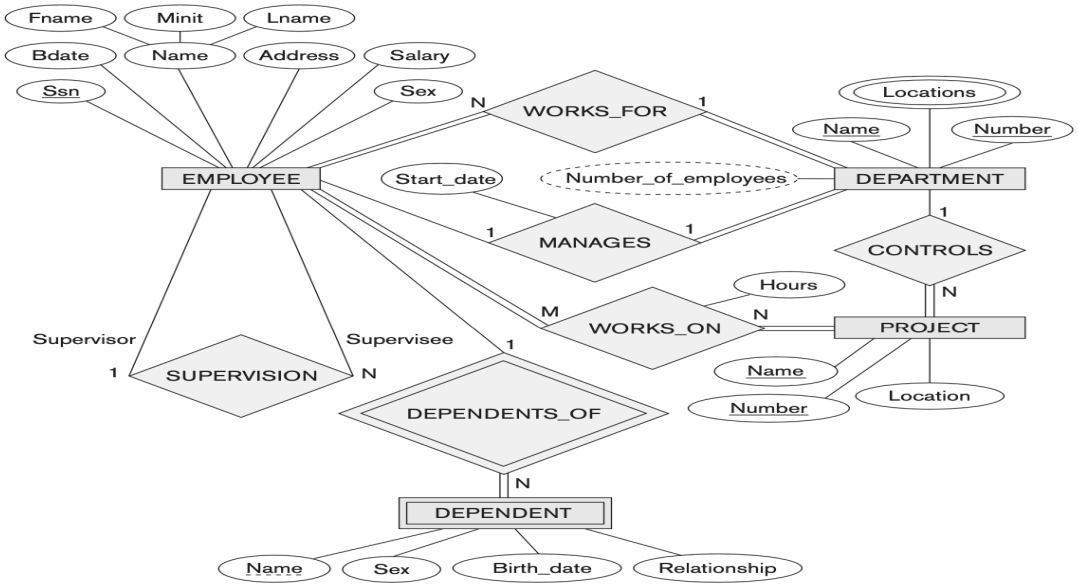
- In ERDs, weak entity type and its identifying relationship are distinguished by surrounding their boxes and diamonds with double lines.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Weak Entity Types - Example

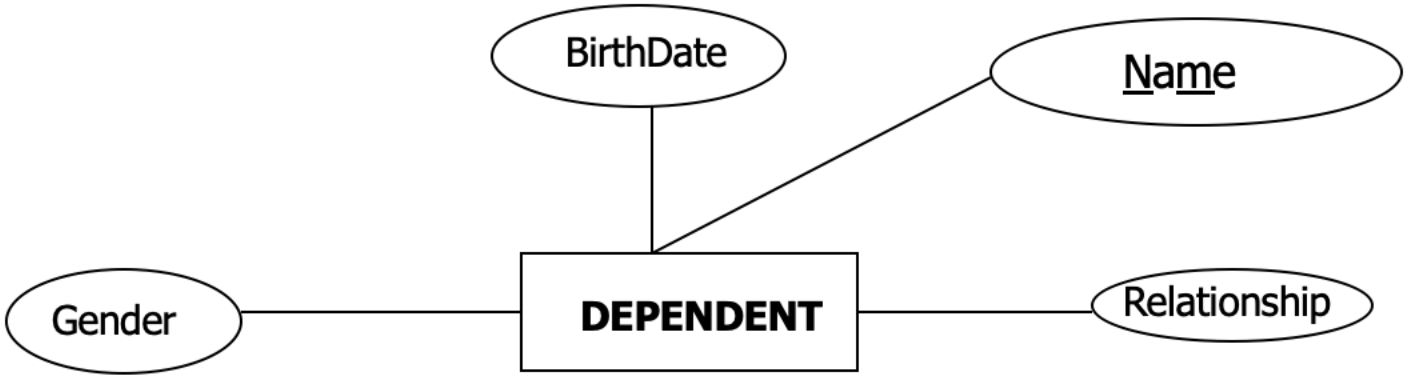
- DEPENDENT
  - Keeps track of the dependents of each employee via a 1:N relationship
  - Attributes: Name, Gender, Birth_Date, Relationship
- Uniqueness of Entities
  - Two dependents of two distinct employees may have the same all attribute values but they are still distinct entities because of being determined by particular employee entity to which each dependent is related to



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Weak Entity Types – Partial Key

- Partial Key
  - A weak entity type has a partial key, which is the attribute that can uniquely identify weak entities in combination with the data from owner/strong entity
  - Ensures that no two dependents have same name, the partial key is attached with the key from strong/owner entity
  - Partial Key is shown as dotted underline.

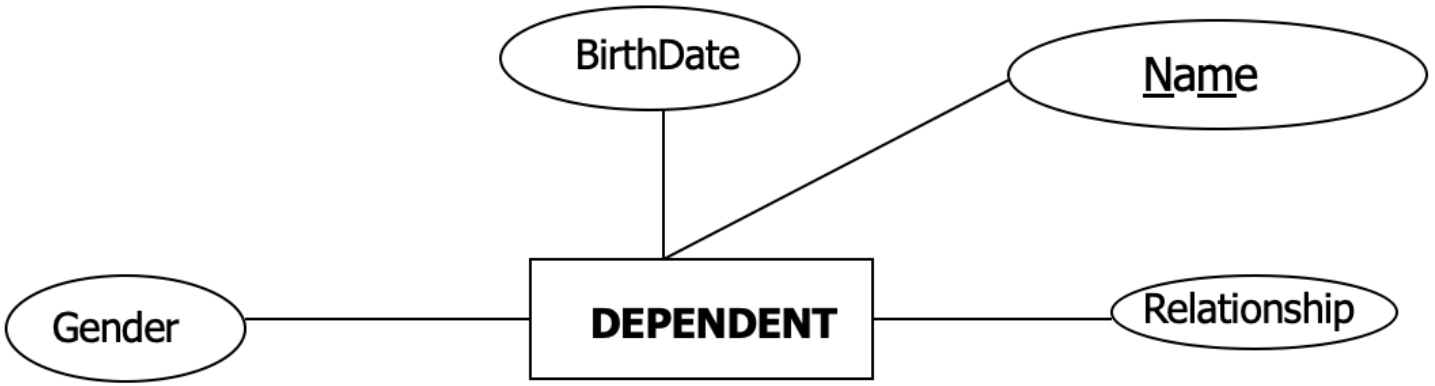


*In worst case, a composite attribute of *all the weak entity's attributes* will be the partial key.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Weak Entity Types – Implementation

- Weak entity types can sometimes be represented as complex (composite, multivalued) attributes.
- Example:
  - we could specify a multivalued attribute Dependents for EMPLOYEE, which is a multivalued composite attribute with the component attributes Name, Birth_date, Gender, and Relationship.
- Or database designer can choose



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Weak Entity Types - Exception

- **Exception**: not every existence dependency results in a weak entity type.
- **Example:** a DRIVER_LICENSE entity cannot exist unless it is related to a PERSON entity, even though it has its own key

  (License_number) and hence is not a weak entity.

- **Observation on Above Exception**:
  - consider the available attributes i.e. DRIVER_LICENSE entity has License_number attribute
  - but any other such entity type having existence dependency may not have such attribute like DEPENDANT do not have attribute like CNIC

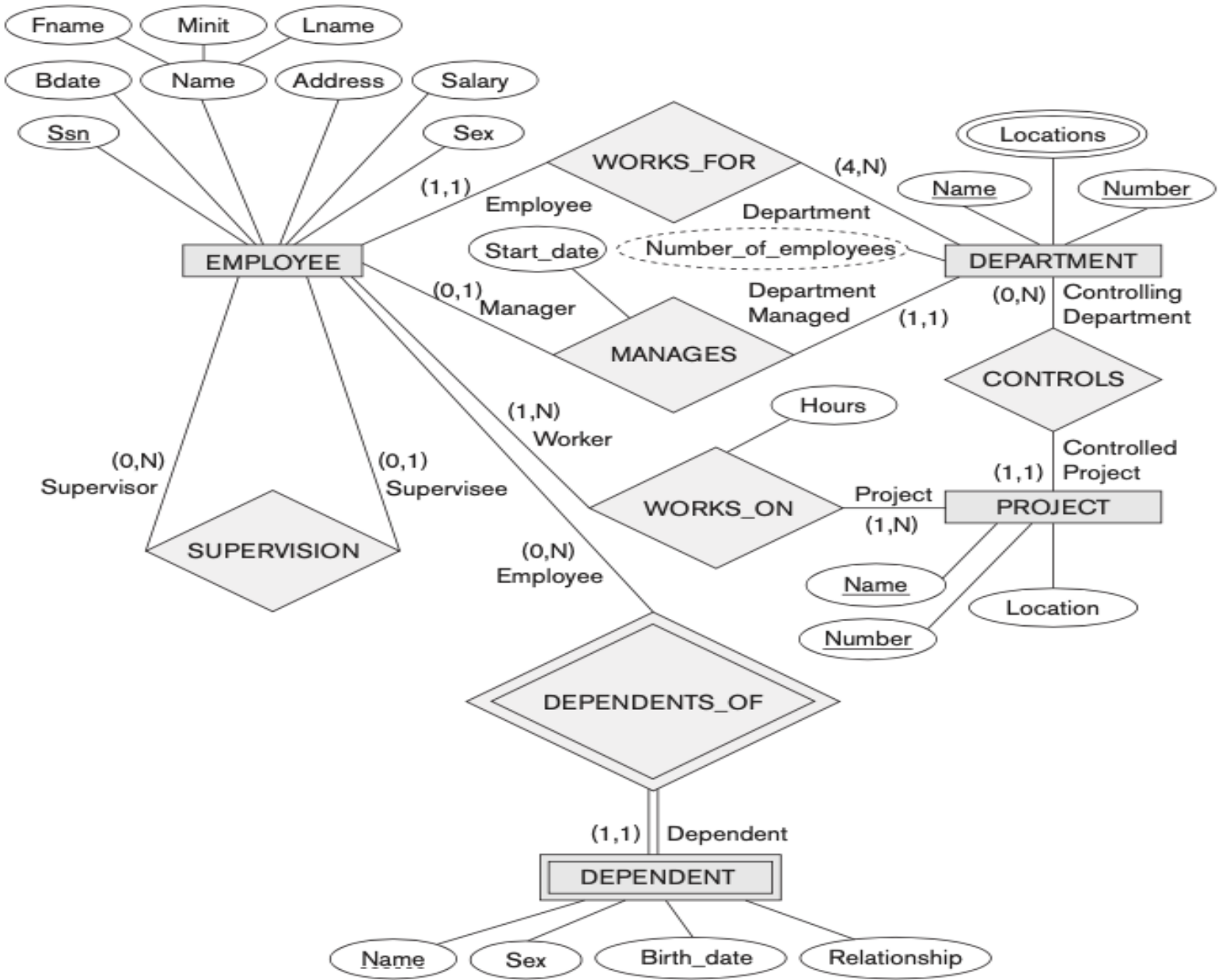
Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Alternative Notations for ER Diagrams – (min,max)

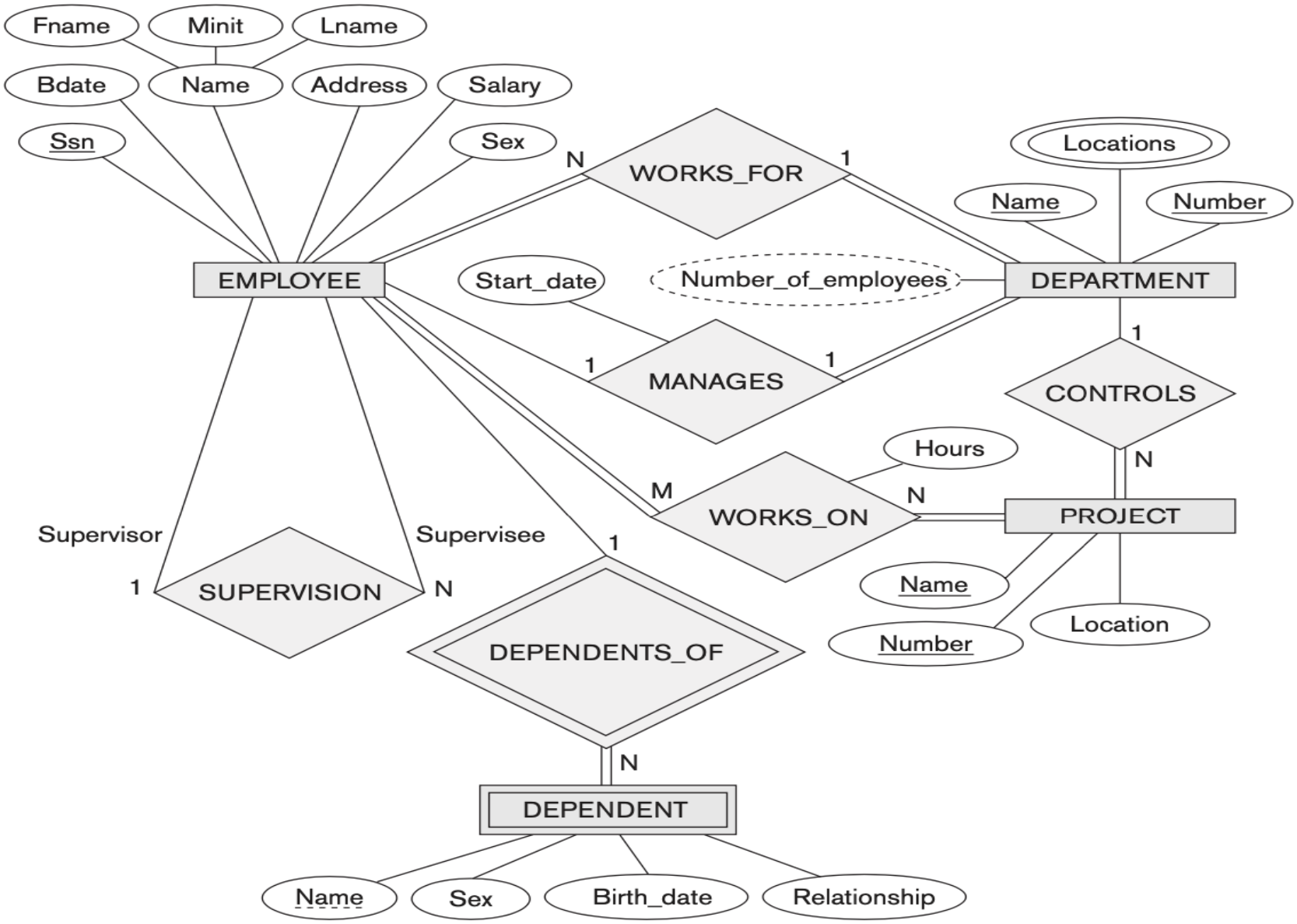
- Associates a pair of integer numbers (min, max) with each *participation* of an entity type *E* in a relationship type *R*
  - where 0 ≤ min ≤ max and max ≥ 1
  - The numbers mean that for each entity *e* in *E*, *e* must participate in at least *min* and at most *max* relationship instances in *R at any point in time*.
  - min = 0 implies partial participation, whereas min > 0 implies total participation.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Alternative Notations for ER Diagrams – with (min,max)



Alternative Notations for ER Diagrams – without (min,max)



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

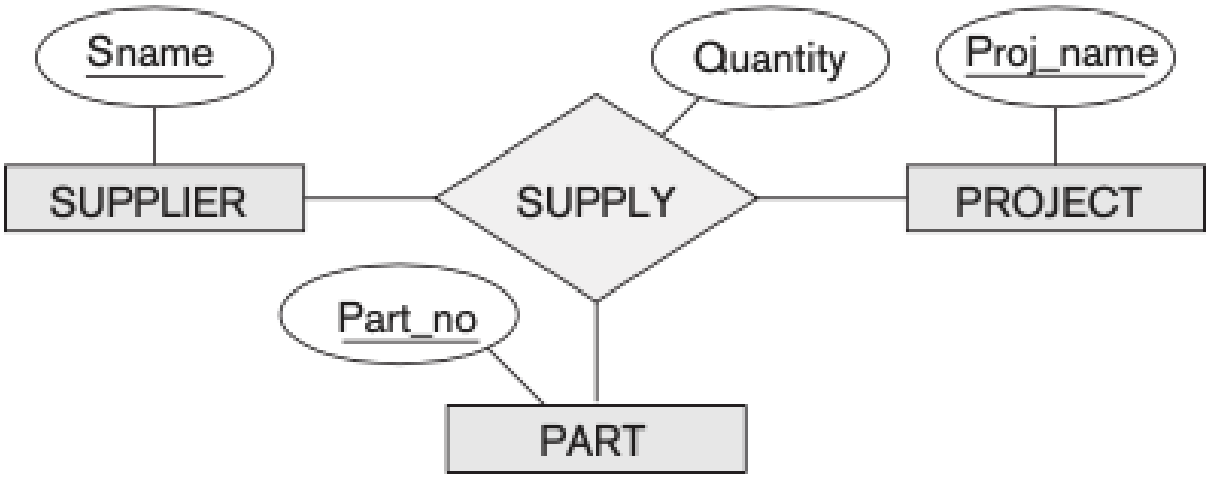
Relationship Types of Degree Higher than Two

- Recap Binary and Ternary Relationships
  - a relationship type of degree two is *binary*
  - relationship type of degree three is *ternary*

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationship Types of Degree Higher than Two

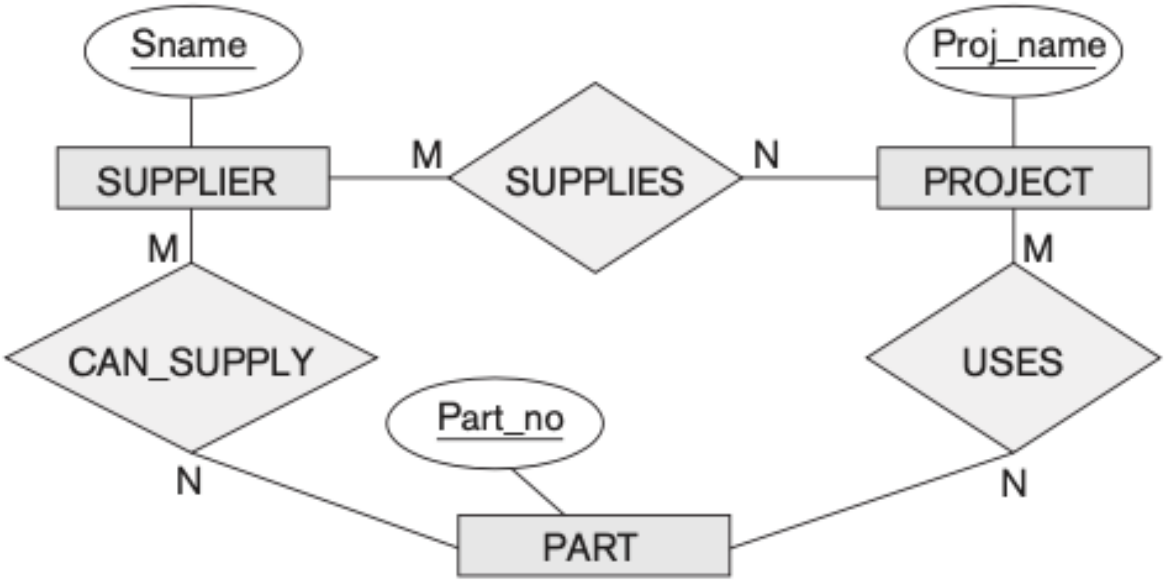
- Example Ternary Relationship
  - SUPPLY is a set of relationship instances (*s*, *j*, *p*), where the meaning is that *s* is a SUPPLIER who is currently supplying a PART *p* to a PROJECT *j*.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationship Types of Degree Higher than Two

- Is below A Ternary Relationship ?
  - The ternary relationship from previous slide is broken as 3 binary relationships



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

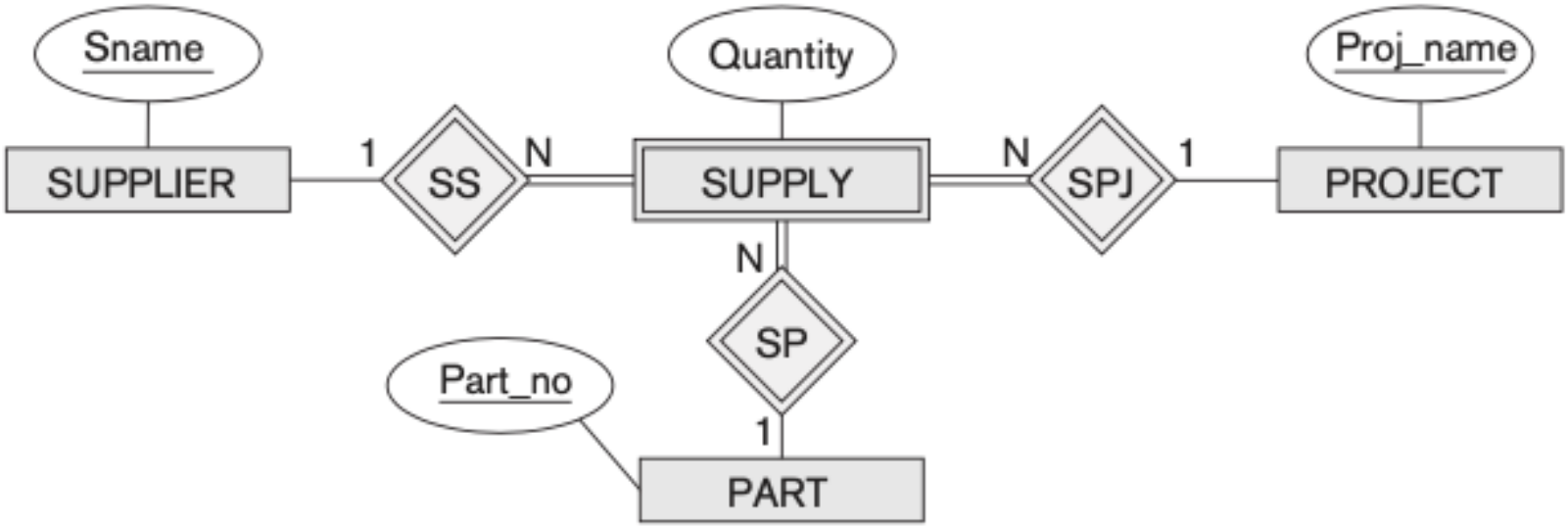
Relationship Types of Degree Higher than Two

- Choosing between Binary and Ternary Relationship
  - often tricky to decide whether a particular relationship should be represented as
    - a relationship type of degree n or should be
    - Or broken down into several relationship types of smaller degrees
- The designer must base this decision on semantics or meaning of the particular situation being represented.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

A Variation – Using Weak Entity Type to Show Ternary as Binary Relationship

- Some database design tools are based on variations of the ER model that permit only binary relationships
  - In this case, a ternary relationship such as SUPPLY must be represented as a weak entity type, with no partial key and with three identifying relationships

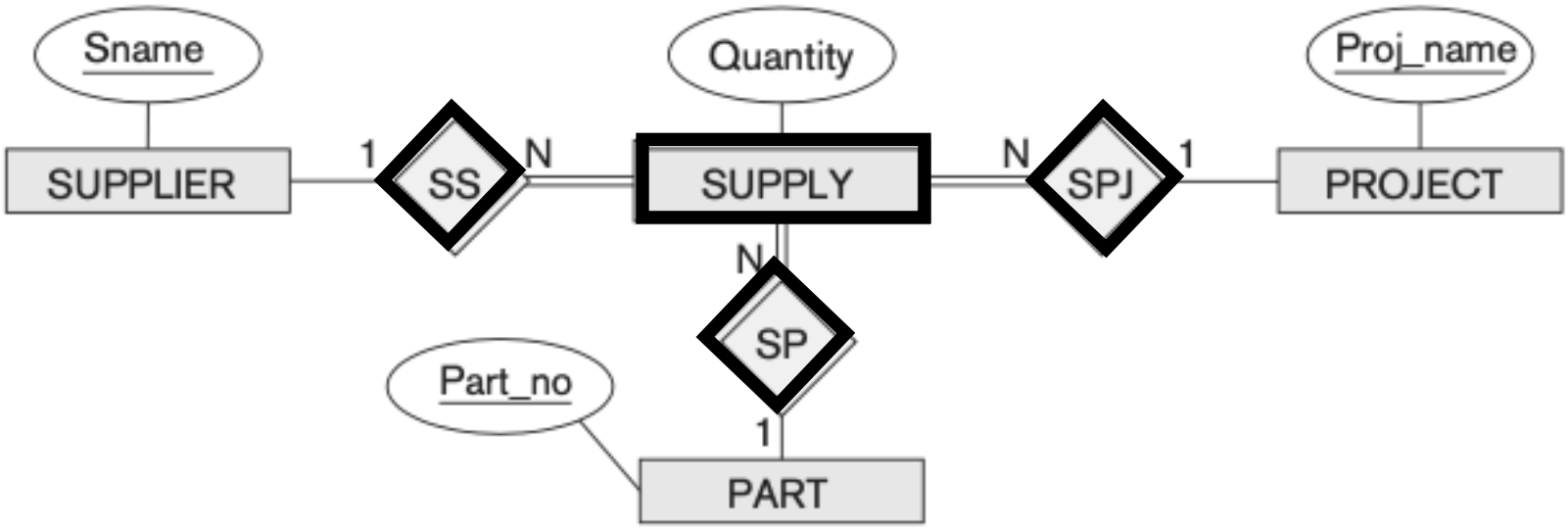


A weak entity is identified by the combination of its three owner entities from SUPPLIER, PART, and PROJECT.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Converting Weak Entity Type into Regular Entity Type

- Add a Surrogate Key
  - An artificially generated value for each entity
- Supply_id could be used for the supply entity type, converting it into a regular entity type.



Supply is now a Regular Entity Type

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

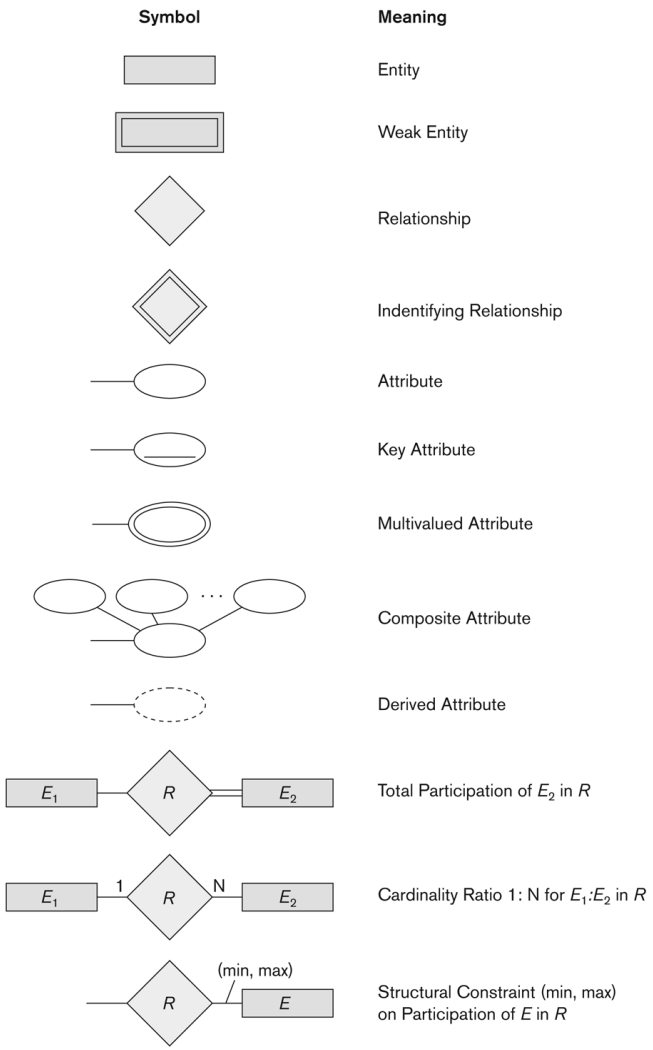
Converting Weak Entity Type into Regular Entity Type

- Refer to other example in figure 3.18 in your text book yourself
  - Page 90

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Summary of notation for ER diagrams

Figure 3.14
Summary of the notation for ER diagrams.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Data Modeling Tools

- A number of popular tools that cover conceptual modeling and mapping into relational schema design.
  - Examples: ERWin, S- Designer (Enterprise Application Suite), ER- Studio, etc.
- POSITIVES:
  - Serves as documentation of application requirements, easy user interface - mostly graphics editor support
- NEGATIVES:
  - Most tools lack a proper distinct notation for relationships with relationship attributes
  - Mostly represent a relational design in a diagrammatic form rather than a conceptual ER-based design

(See Chapter 12 for details)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Some of the Currently Available Automated Database Design Tools

COMPANY	TOOL	FUNCTIONALITY
Embarcadero Technologies	ER Studio	Database Modeling in ER and IDEF1X
	DB Artisan	Database administration, space and security management
Oracle	Developer 2000/Designer 2000	Database modeling, application development
Popkin Software	System Architect 2001	Data modeling, object modeling, process modeling, structured analysis/design
Platinum (Computer Associates)	Enterprise Modeling Suite: Erwin, BPWin, Paradigm Plus	Data, process, and business component modeling
Persistence Inc.	Pwertier	Mapping from O-O to relational model
Rational (IBM)	Rational Rose	UML Modeling & application generation in C++/JAVA
Resolution Ltd.	Xcase	Conceptual modeling up to code maintenance
Sybase	Enterprise Application Suite	Data modeling, business logic modeling
Visio	Visio Enterprise	Data modeling, design/reengineering Visual Basic/C++

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Extended Entity-Relationship (EER) Model (in next chapter)

- The entity relationship model in its original form did not support the specialization and generalization abstractions
- Next chapter illustrates how the ER model can be extended with
  - Type-subtype and set-subset relationships
  - Specialization/Generalization Hierarchies
  - Notation to display them in EER diagrams

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Chapter Summary

- ER Model Concepts: Entities, attributes, relationships
- Constraints in the ER model
- Using ER in step-by-step conceptual schema design for the COMPANY database
- ER Diagrams - Notation
- Alternative Notations – UML class diagrams, others

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei